



Islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CU6051NI Artificial Intelligence

75% Individual Coursework

Submission: Final Submission

Academic Semester: Autumn Semester 2025

Credit: 15 credit semester long module

Student Name: Resha Shrestha

London Met ID: 23048661

College ID: NPO1AI230121

Assignment Due Date: 21/01/2026

Assignment Submission Date: 21/01/2026

Submitted To: Mr. Alish KC

GitHub Link	https://github.com/ReshaShrestha2004/Diabetes-Prediction-System
--------------------	---

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table Of Contents

1. INTRODUCTION.....	7
1.1 Introduction to AI in Healthcare.....	7
1.2 Introduction to chosen problem domain/topic.....	7
1.3 Introduction to Project.....	9
2. BACKGROUND.....	11
2.1 Literature Review and Existing Work in Diabetes Prediction Using Machine Learning.....	13
2.2 Analysis of the Reviewed Literature.....	20
3. SOLUTION.....	21
3.1 Related Theory.....	21
3.1.1 Logistic Regression.....	21
3.1.2 Random Forest.....	24
3.1.3 XGBoost.....	27
3.2 PSEUDOCODE.....	30
3.2.1 Pseudocode for Logistic Regression.....	30
3.2.2 Pseudocode for Random Forest.....	31
3.2.3 Pseudocode for XGBoost.....	32
3.3. Diagrammatical representations of the solution.....	33
3.3.1 Proposed Model Diagram.....	33
3.3.2 Flowcharts.....	34
3.3.3 State Transition Diagram.....	38
3.4 Tools and Technologies Used.....	39
3.4.1 Programming Language.....	39
3.4.2 Data Management and Numerical Computing.....	40

3.4.3 Data Visualization	41
3.4.3 Machine Learning Frameworks	42
3.4.3 Development Environment	43
3.5 Development Process.....	44
3.5.1 Data Loading	44
3.5.2. Data Cleaning	45
3.5.3. Exploratory Data Analysis	46
3.5.4. Data Pre-processing	49
3.5.5. Model Definition with class weights	53
3.5.6. Cross Validation	57
3.5.7. Best Model selection	59
3.5.8. Final Model Training	62
3.5.9. Model Evaluation	63
3.6 Comparative Analysis & Insights	69
4. CONCLUSION	75
4.1 Analysis of Work Done	75
4.2 Real-World Applications.....	76
4.3 Future Work.....	76
4.4 Final Remarks	77
Bibliography	78

TABLE OF FIGURES

Figure 1 Type 2 diabetes mellitus risk factors and complications.....	8
Figure 2 Relative risks of type 2 diabetes according to joint levels of physical activity, BMI and glucose homoeostasis.....	11
Figure 3 Flow chart outlining steps in the clinical diagnosis of diabetes mellitus. OGTT Oral glucose tolerance test.....	12
Figure 4 Sigmoid Function (Anon., 2025).....	22
Figure 5 An illustration of a random forest (Pijush Dutta, 2021)	24
Figure 6 XGBoost (Anon., 2025)	27
Figure 7 Proposed Model Diagram.....	33
Figure 8 System Architecture Flowchart.....	34
Figure 9 Logistic Regression Flowchart	35
Figure 10 Random Forest Flowchart.....	36
Figure 11 XGBoost Flowchart	37
Figure 12 State Transition Diagram.....	38
Figure 13 Python	39
Figure 14 Pandas	40
Figure 15 NumPy	40
Figure 16 Matplotlib.....	41
Figure 17 Seaborn.....	41
Figure 18 Scikit learn.....	42
Figure 19 Imbalanced-learn	42
Figure 20 XGBoost.....	43
Figure 21 Google Colab	43
Figure 22 Data Loading.....	44
Figure 23 Data Cleaning	45
Figure 24 Class Distribution Analysis	46
Figure 25 Correlation Heatmap for features	46
Figure 26 Outlier Detection Histplots.....	47
Figure 27 Box plots	48

Figure 28 Data Scaling Procedure	49
Figure 29 Data Scaling Result.....	50
Figure 30 Train Test Split	50
Figure 31 Class Balancing SMOTE procedure.....	51
Figure 32 Class Balancing SMOTE result.....	51
Figure 33 Class Balancing SMOTE visualizations	52
Figure 34 Defining class weights.....	53
Figure 35 Binary model definition.....	53
Figure 36 Defining class weights multiclass	53
Figure 37 Multiclass model definition	54
Figure 38 Hyperparameters adjustment table	56
Figure 39 Cross Validation Procedure.....	57
Figure 40 Cross-Validation Result.....	58
Figure 41 Best Model selection binary	59
Figure 42 Best Model selection multiclass	60
Figure 43 Final Model Training.....	62
Figure 44 Classification report for binary classification.....	63
Figure 45 Confusion matrix for binary classification	63
Figure 46 Classification report for multiclass classification.....	65
Figure 47 Confusion matrix for multiclass classification	65
Figure 48 ROC-AUC Curve for binary classification.....	67
Figure 49 Sample Predictions	68

TABLE OF TABLES

Table 1 Literature Matrix	19
Table 2 Analysis of the Reviewed Literature	20
Table 3 Binary Classification Comparative Analysis	70
Table 4 Multiclass Classification Comparative Analysis	72

TABLE OF EQUATIONS

Equation 1 Logistic Regression Function (Anon., n.d.)	21
Equation 2 Softmax Function (Anon., 2025).....	22
Equation 3 Multinomial Logistic Regression (Anon., 2025)	23
Equation 4 Mean Squared Error (Schott, 2019)	24
Equation 5 Gini Impurity (Schott, 2019).....	25
Equation 6 Entropy (Schott, 2019)	25
Equation 7 XGBoost Model (XGBoost, 2025)	27
Equation 8 objective function in XGBoost (XGBoost, 2025).....	28
Equation 9 15 Additive Training (Boosting Framework) (XGBoost, 2025).....	28
Equation 10 Information gain (XGBoost, 2025).....	28

1. INTRODUCTION

1.1 Introduction to AI in Healthcare

Artificial intelligence (AI) is the ability of a digital computer or robot controlled by a computer to perform tasks usually done by intelligent beings (Copeland, 2025). This term is commonly used when talking about creating systems that have human-like intellectual processes, such as reasoning, discovering meaning, generalizing, and learning from past experiences (Copeland, 2025). Since they were first created in the 1940s, digital computers have been programmed to handle very complex tasks like proving mathematical theorems or playing chess with impressive skill (Copeland, 2025). Even though there have been continuous advancements in computer processing speeds and memory, no programs have yet matched the full adaptability of human intelligence in wider areas or in tasks that require extensive everyday knowledge. On the other hand, some programs have achieved performance levels similar to human experts in specific tasks, showing that artificial intelligence, in this narrower sense, is found in many applications, such as medical diagnosis, computer search engines, voice or handwriting recognition, and chatbots (Copeland, 2025). The growing access to public health datasets has led to significant research in predictive analytics for chronic diseases, such as diabetes.

1.2 Introduction to chosen problem domain/topic

Diabetes mellitus is a long-lasting metabolic condition mainly defined by elevated blood glucose levels and is linked to significant long-term complications that impact the cardiovascular system, kidneys, nerves, and various other organs (Saruar Alam, 2021). Common symptoms seen in diabetic patients often include extreme thirst and hunger, frequent urination, tiredness, blurred vision, and weight loss (Saruar Alam, 2021). Diabetes is generally classified into two main types: type 1 diabetes and type 2 diabetes (Saruar Alam, 2021). Type 1 diabetes is marked by a significant lack or absence of insulin production by the pancreas due to an unknown immune system disorder, and it tends to affect younger individuals, including children (Saruar Alam, 2021). The main treatment for type 1 diabetes is insulin therapy. Type 2 diabetes is defined by insulin resistance,

where the body produces insulin but does not use it effectively to control blood sugar levels (Marwa Mustafa Owess, 2024).

If not identified early, it can lead to serious issues like heart disease, kidney failure, nerve damage, and a lower quality of life. Due to the increase in sedentary habits, poor diets, and rising obesity rates, diabetes has emerged as a major public health challenge today (Poudel, 2024). Therefore, it is essential to identify at-risk individuals promptly, as early interventions like lifestyle changes, medical supervision, and behavioral adjustments can greatly lower the chances of developing Type 2 diabetes (Samantha Roberts 1, 2017).

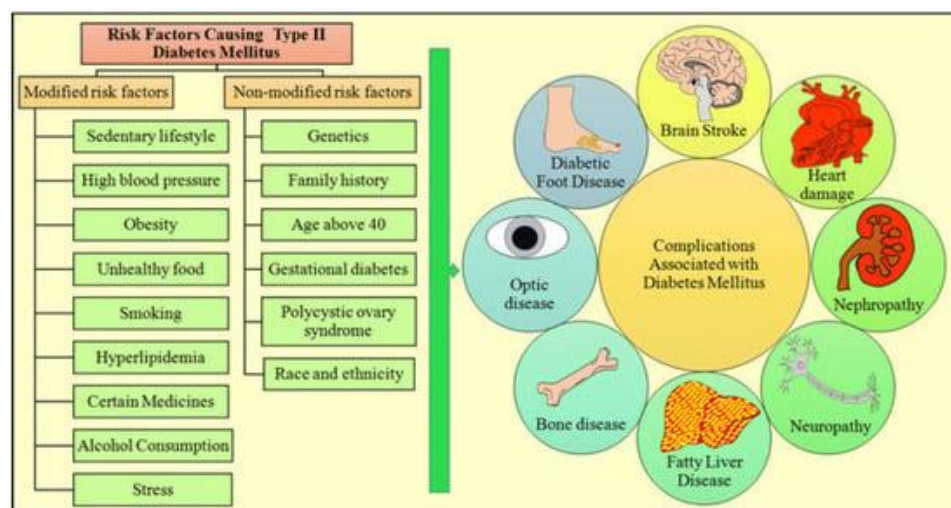


Figure 1 Type 2 diabetes mellitus risk factors and complications

(Neha Tiwary, 2023)

Traditional methods for evaluating diabetes risk typically depend on clinical lab tests, which may not always be practical or cost-effective for large-scale screenings.

In contrast, machine learning provides a scalable and efficient alternative by using self-reported health indicators, lifestyle habits, and demographic data to estimate diabetes risk. These methods enable wider and less resource-intensive risk assessments, making them important tools for public health monitoring and preventive healthcare strategies.

1.3 Introduction to Project

This project focuses on creating machine learning models to predict diabetes risk using two versions of the CDC Diabetes Health Indicators dataset from the UCI Machine Learning Repository.

- The first dataset is binary-class, with the target variable having two labels, 0 (No diabetes) and 1 (Diabetes).
- The second dataset is multiclass, featuring three labels, 0 (No diabetes), 1 (Prediabetes), and 2 (Diabetes) which allows for a more detailed and realistic classification task.

By using both datasets separately, the project aims to evaluate how different machine learning algorithms perform at various complexity levels, leading to a better understanding of model behavior in both binary and multiclass classification scenarios. This approach with two datasets provides a more comprehensive analysis of diabetes prediction, improves comparative studies, and aligns with real-world healthcare situations where patients can be placed into multiple risk categories instead of just a simple positive/negative label. Both datasets contain 21 health-related features, including

- Body Mass Index (BMI)
- Physical activity
- Blood pressure
- Cholesterol levels
- General health ratings
- Demographic details.

The algorithms utilized in this research are classified as supervised learning, as each model is trained with labeled data. Supervised learning is a kind of machine learning in which a model learns from labeled data, this means that each input has a matching correct output and then the model predicts outcomes and checks them against the actual outputs, modifying itself to minimize mistakes and enhance accuracy over time to make precise predictions on new, previously unseen data (Anon., 2025).

In particular, three supervised classification algorithms are used:

- Logistic Regression – a linear and easily interpretable classification model.
- Random Forest – an ensemble of decision trees based on bagging that decreases variance and improves stability.
- XGBoost (Extreme Gradient Boosting) – a sophisticated boosting framework recognized for its excellent predictive performance on structured datasets.

By training and assessing all three algorithms on the same dataset, this project enables a comparative analysis that showcases their strengths, weaknesses, interpretability, and appropriateness for predicting diabetes at the population level. The study also aids in understanding how lifestyle and behavioral indicators can be utilized to create early warning systems for chronic diseases. The ultimate goal is to determine which model provides the best balance between accuracy, robustness, and practical deployment potential.

2. BACKGROUND

Diabetes mellitus (DM) is a metabolic disorder characterized by high blood glucose levels (Sapra & Bhandari., 2023). DM includes several types, such as type 1, type 2, maturity-onset diabetes of the young (MODY), gestational diabetes, neonatal diabetes, and secondary causes from conditions like endocrinopathies or steroid use (Sapra & Bhandari., 2023). The primary subtypes are Type 1 diabetes mellitus (T1DM) and Type 2 diabetes mellitus (T2DM), which are caused by defective insulin secretion (T1DM) and/or action (T2DM) (Sapra & Bhandari., 2023). T1DM typically occurs in children or adolescents, while T2DM usually affects middle-aged and older adults with long-term high blood sugar due to unhealthy lifestyle and dietary habits (Sapra & Bhandari., 2023). The causes and development of T1DM and T2DM are quite different, leading to various causes, symptoms, and treatments for each type (Sapra & Bhandari., 2023).

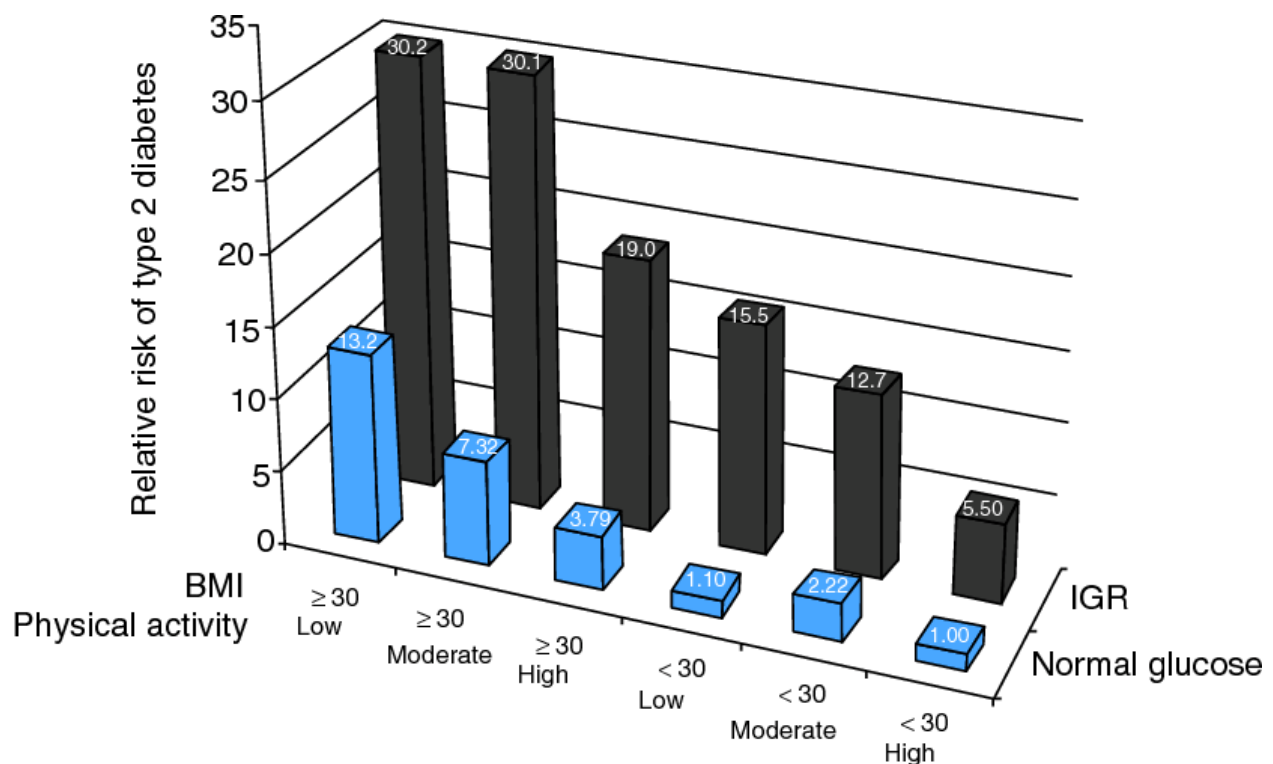


Figure 2 Relative risks of type 2 diabetes according to joint levels of physical activity, BMI and glucose homeostasis

(G. Hu, 2006)

The rise in Type 2 diabetes cases is closely associated with lifestyle choices like lack of exercise, poor diet, obesity, smoking, and high stress levels. Consequently, researchers

are increasingly using data-driven methods to improve the early detection of at-risk individuals. Conventional diabetes diagnosis depends on biochemical tests such as fasting glucose levels, oral glucose tolerance tests, and HbA1c readings. While these methods are precise, they can be slow, expensive, and hard to access for people in remote or under-resourced areas.

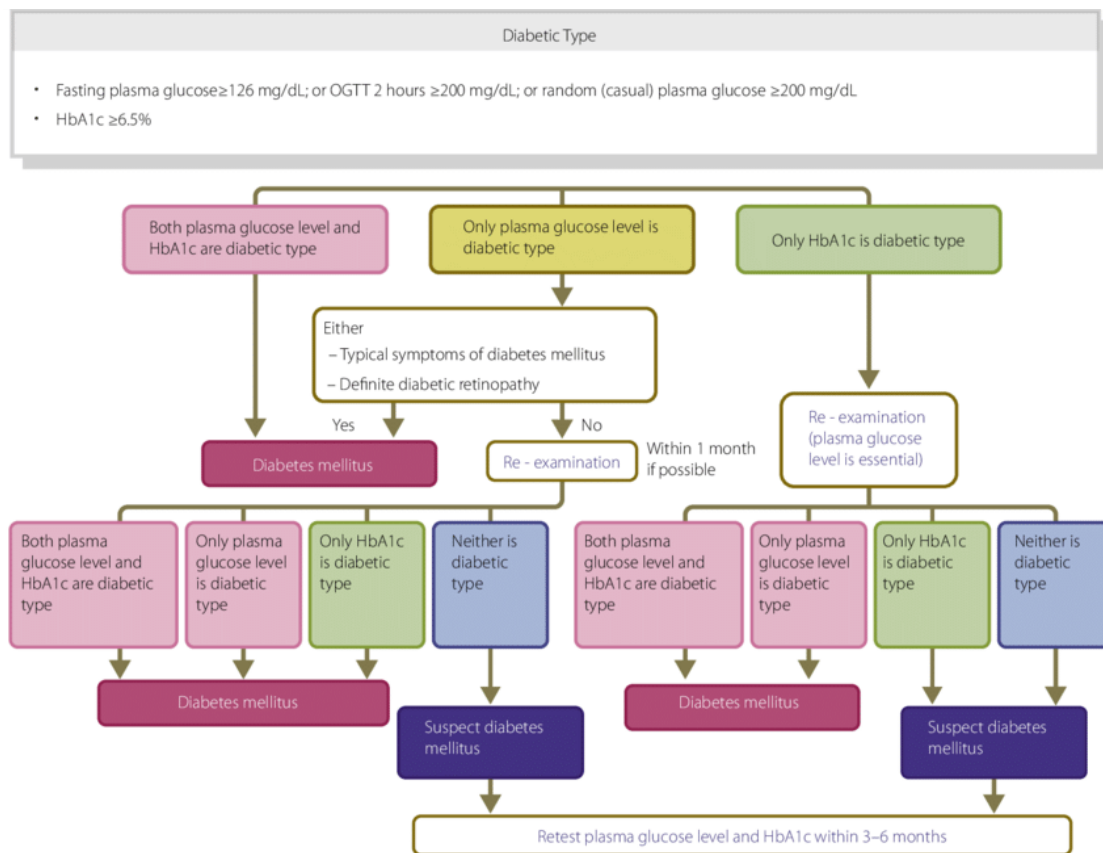


Figure 3 Flow chart outlining steps in the clinical diagnosis of diabetes mellitus. OGTT Oral glucose tolerance test.

(Haneda, 2018)

This issue has led to the incorporation of Artificial Intelligence (AI) and Machine Learning (ML) into preventive healthcare systems. A significant amount of research has investigated the application of machine learning methods for predicting diabetes, utilizing both clinical data and extensive health surveys. Current studies assess various algorithms ranging from traditional statistical models to contemporary ensemble methods emphasizing the advantages and drawbacks of each technique.

The section 2.1 examines five pertinent research articles, concentrating on the datasets employed, the algorithms used, and the outcomes obtained. These studies together guide the methodological decisions in this project

2.1 Literature Review and Existing Work in Diabetes Prediction Using Machine Learning.

In the last few years, research on using machine learning to predict diabetes has expanded significantly. Numerous studies indicate that computational models can detect individuals at high risk sooner than traditional clinical approaches.

Paper 1: Smith et al. (1988) – Early ML for Diabetes Prediction

Algorithm Used: ADAP Learning Algorithm (a predecessor to neural networks)

Dataset: Pima Indians Diabetes Dataset (UCI)

Results: Showed moderate predictive accuracy, indicating that ML can assist in medical diagnosis.

Summary:

This important study presented one of the first machine learning methods for predicting diabetes. By utilizing the Pima dataset, the authors employed the ADAP algorithm to predict diabetes onset based on clinical features. While the accuracy was not as high as that of current algorithms, the research demonstrated the potential of using AI for disease prediction. This study established a foundation for future investigations in medical classification tasks. (al, 1988)

Paper 2: Sisodia & Sisodia (2018) – Classical ML Algorithms for Diabetes

Algorithms Used:

- Decision Tree
- SVM
- Naïve Bayes

Dataset: Pima Indians Diabetes Dataset

Results: Naive Bayes classification algorithm outperforms comparatively other algorithms with accuracy of 76.30 %.

Summary:

The authors evaluated various traditional machine learning classifiers for predicting diabetes. Their findings showed that Naive Bayes machine learning classifier can predict the chances of diabetes with more accuracy as compared to other classifiers. The research highlights the significance of choosing the right algorithm based on the characteristics of the dataset.

(Deepti Sisodia, 2018)

Paper 3: Gong et al. (2019) — A Comparative Study of Machine Learning Approaches for Diabetes Risk Prediction: Insights from SHAP and Feature Importance

Dataset: BRFSS health indicators

Algorithms:

- Logistic Regression
- Random Forest
- XGBoost

Results: XGBoost had the best AUC (0.83)

Additional Findings: SHAP analysis identified general health, blood pressure status, and BMI as key predictors.

Summary:

This research utilized three supervised learning models to forecast diabetes using BRFSS data. XGBoost surpassed all other models, showing a strong capability to identify complex patterns in health and lifestyle indicators. The application of SHAP for interpretability enhanced the study by highlighting the factors that most significantly influence diabetes risk.

(Yiru Gong, 2024)

Paper 4 — Al-Mallah et al. (2016) – Predicting diabetes mellitus using SMOTE and ensemble machine learning approach: The Henry Ford Exercise Testing (FIT) project

Dataset: Cardiorespiratory fitness records from 32,555 patients (Henry Ford Health System, 1991–2009)

- Algorithms Used:
- Naïve Bayes
- Logistic Regression
- Logistic Model Tree (LMT)
- Decision Tree
- Random Forest
- Ensemble model (Voting across NB Tree, RF, LMT)

Handling Class Imbalance: SMOTE (Synthetic Minority Oversampling Technique)

Results:

- Ensemble classifier achieved AUC = 0.92, outperforming individual algorithms.
- Best predictors were selected using clinical importance, multiple linear regression analysis, and information gain.
- Demonstrated that cardiorespiratory fitness indicators can strongly predict incident diabetes.

Summary:

Al-Mallah et al. (2016) conducted a large-scale study exploring the use of machine learning for predicting incident diabetes using cardiorespiratory fitness data. The dataset included 32,555 patients with no prior diagnosis of heart failure or coronary artery disease, all of whom completed treadmill stress tests and were tracked for five years. Among these participants, 5,099 eventually developed diabetes. The study evaluated several machine learning algorithms including Logistic Regression, Naïve Bayes, Random Forest, and Logistic Model Trees and selected the most relevant predictors using statistical and information-based feature ranking methods. To address the

imbalance between diabetic and non-diabetic cases, the authors applied SMOTE, which significantly improved model performance. Their final ensemble model, which combined Naïve Bayes Tree, Random Forest, and Logistic Model Tree via a voting strategy, achieved exceptionally strong performance (AUC = 0.92). This study highlights the effectiveness of ensemble learning and oversampling techniques for building robust diabetes prediction systems.

(Manal Alghamdi, 2017)

Paper 5 — Supervised Machine Learning-Based Models for Predicting Raised Blood Sugar

Dataset: WHO STEPwise Approach to NCD Risk Factor Surveillance (Palestinian Adults)

Algorithms Used:

- Random Forest
- Bagging Decision Trees
- XGBoost
- AdaBoost
- Decision Tree
- Multi-Layer Perceptron (MLP)

Results:

- Random Forest achieved the highest accuracy: 98.4%.
- Bagging Decision Trees: 97.4%
- XGBoost: 96.4%
- MLP: 96.3%
- AdaBoost: 95.2%
- Decision Tree: 94.8%

Summary:

This research aims to predict elevated blood sugar, an important early sign of prediabetes and diabetes, using supervised machine learning models trained on the WHO STEPwise NCD risk-factor dataset gathered from adult participants in the Palestinian community. The dataset contains crucial diabetes-related risk factors such as age, BMI, physical activity, dietary habits, prevalence of comorbidities, and fasting blood sugar levels. These factors were utilized to train various classical and ensemble machine-learning algorithms. Among the models assessed, Random Forest showed the best predictive accuracy at 98.4%, followed by Bagging Decision Trees and XGBoost. The findings indicate that ensemble-based models, especially Random Forest and bagging methods, are very effective for the early identification of high blood sugar levels in health datasets at the

population level. This study emphasizes the importance of machine learning for scalable screening in communities where early detection can significantly lessen the long-term impact of diabetes.

(Marwa Mustafa Owess, 2024)

Literature Matrix

Study	Dataset	Algorithms	Best Model	Accuracy/ AUC	Key Findings
Smith et al.	Pima Indians (UCI)	ADAP Learning Algorithm	ADAP	Moderate	First ML for diabetes prediction; foundational work
Sisodia & Sisodia	Pima Indians	Decision Tree, SVM, Naïve Bayes	Naïve Bayes	76.30%	Classical algorithms comparison; algorithm selection matters
Gong et al.	BRFSS	Logistic Regression, Random Forest, XGBoost	XGBoost	AUC = 0.83	SHAP analysis; key predictors: health, BP, BMI
Al-Mallah et al.	Henry Ford FIT (n=32,555)	NB, LR, LMT, DT, RF, Ensemble	Ensemble Voting	AUC = 0.92	SMOTE used; ensemble outperforms individual models
Owess et al.	WHO STEPS (Palestine)	RF, Bagging, XGBoost, AdaBoost, DT, MLP	Random Forest	98.4%	Early detection focus; ensemble methods highly effective

Table 1 Literature Matrix

2.2 Analysis of the Reviewed Literature

Table 2 Analysis of the Reviewed Literature

Study	Diagnostic Definition	ML Methods	Key Techniques/Tools	Performance Highlights
Smith al, 1988	Lab-confirmed glucose tests	Traditional algorithms	Basic classification	Foundational work with limited datasets
Deepti Sisodia, 2018	Fasting glucose, oral glucose tolerance b tests	Naïve Bayes, Decision Tree, SVM	Traditional diagnostic labels	Focus on conventional algorithms for diabetes risk prediction
Yiru Gong, 2024	Self-reported diabetes indicators	Random Forest, XGBoost	SHAP value analysis for interpretability	Impressive XGBoost results, better predictive accuracy
Manal Alghamdi, 2017	Diabetes incidence over extended follow-up	Ensemble models	SMOTE for class imbalance, feature importance	Ensemble models greatly outperform individual classifiers
WHO STEPS (Palestinian adults)	Elevated blood sugar, early-stage hyper-glycemia	Random Forest, ensemble methods	Early detection screening, data balancing	Notably high Random Forest accuracy, better screening tool
Overall Pattern	Shift from lab-confirmed to self-reported and early detection	Progression from traditional to ensemble	SMOTE, SHAP, feature-importance tools	Ensemble methods (RF, XGBoost, boosting) consistently outperform linear classifiers; interpretability essential

3. SOLUTION

3.1 Related Theory

3.1.1 Logistic Regression

Logistic regression in machine learning is an effective and straightforward algorithm that examines the connection between one or more independent variables and sorts data into various classes and is commonly used in predictive modeling to predict the likelihood that a particular instance belongs to a certain category (Saharawat, 2025).

Types of Logistic Regression:

1. Binomial Logistic Regression: This type is applied when the dependent variable has just two possible categories (Saharawat, 2025)
2. Multinomial Logistic Regression: This is applied when the dependent variable has three or more possible categories that are not arranged in order (Saharawat, 2025)
3. Ordinal Logistic Regression: This type is used when the dependent variable has three or more categories that have a natural order or ranking (Saharawat, 2025)

Mathematically in binary logistic regression:

Logistic regression applies the sigmoid function to continuous value in order to convert it into a probability between 0 and 1 which can be used to predict the class it belongs to.

$$f(x) = \frac{1}{1 + e^{-x}}$$

Equation 1 Logistic Regression Function (Anon., n.d.)

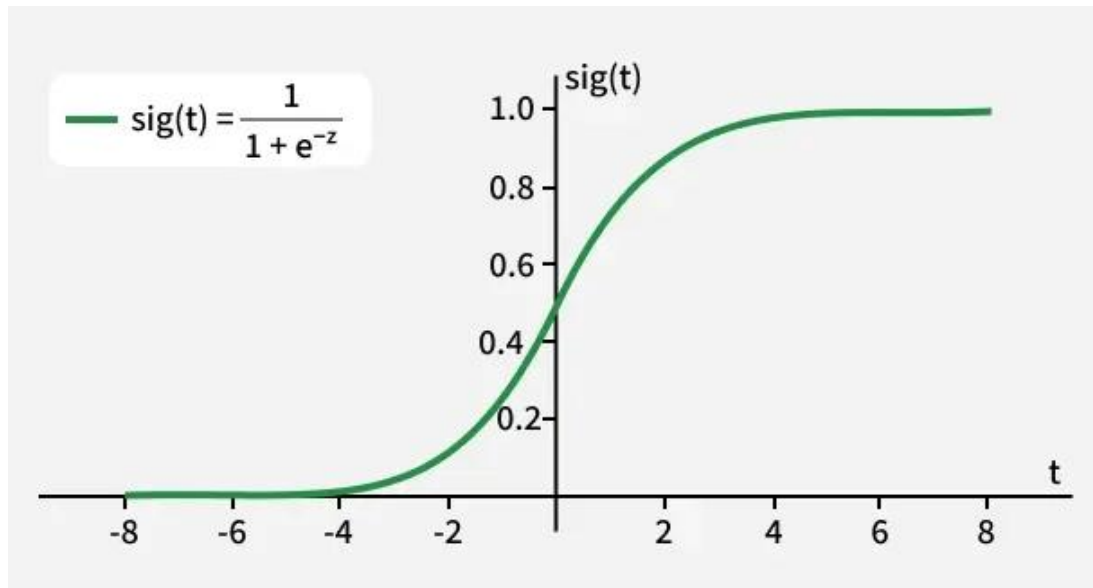


Figure 4 Sigmoid Function (Anon., 2025)

The sigmoid function, as illustrated above, transforms continuous variable data into probabilities ranging from 0 to 1.

$f(x)$ approaches 1 as z approaches infinity (Anon., 2025)

$f(x)$ approaches 0 as z approaches negative infinity (Anon., 2025)

$f(x)$ is always confined between 0 and 1 (Anon., 2025)

The probability of belonging to a class can be expressed as:

$$P(y=1) = \sigma(z)$$

$$P(y=0) = 1 - \sigma(z) \text{ (Anon., 2025)}$$

Mathematically in multinomial logistic regression:

The softmax function is used in place of the sigmoid function.

$$\text{softmax}(z_i) = \frac{e^{z_j}}{\sum_{j=1}^K e^{z_j}}$$

Equation 2 Softmax Function (Anon., 2025)

Here, K stands for the number of elements in the vector z and i, j goes through all the elements in the vector.

Then, the probability for class c will be:

$$P(Y = c | \vec{X} = x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{k=1}^K e^{w_k \cdot x + b_k}}$$

Equation 3 Multinomial Logistic Regression (Anon., 2025)

3.1.2 Random Forest

Random Forest is an ensemble model based on bagging, made up of several decision trees that are trained on bootstrapped subsets of the data. Each tree casts a vote, and the final prediction is determined by the majority vote. This method employs the principle of decision trees, creating a set of decision trees and combining their results to produce the final prediction. Each decision tree within a random forest is built using random data subsets, and every single tree is trained on a segment of the complete dataset. Afterward, the results from all decision trees are merged to obtain the final forecast (Salman, 2024).

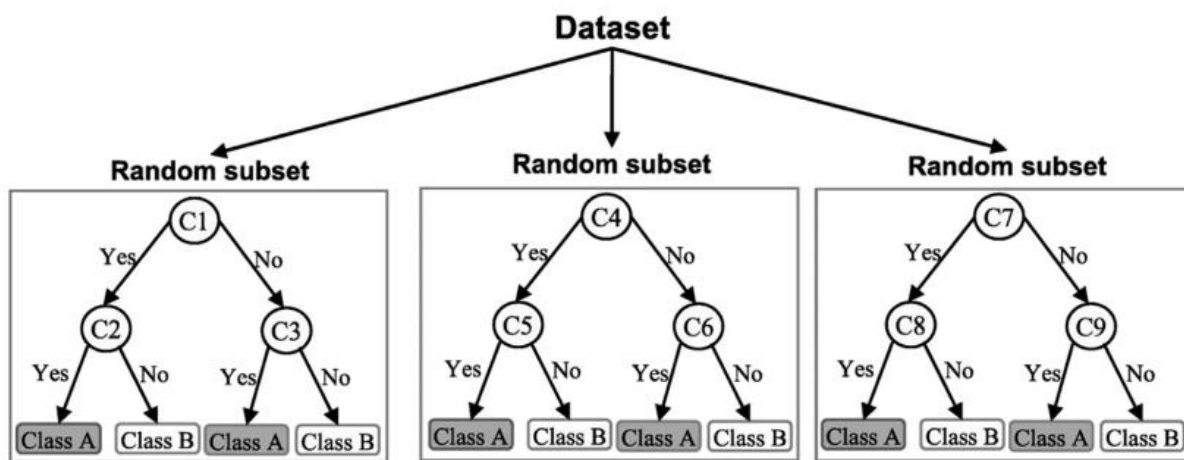


Figure 5 An illustration of a random forest (Pijush Dutta, 2021)

Mathematics behind random forest:

A. Regression Issues (using Mean Squared Error — MSE)

When Random Forest is applied for regression, each decision tree split is selected to reduce the Mean Squared Error (MSE) (Schott, 2019).

$$MSE = \frac{1}{N} \sum_{i=1}^N (f_i - y_i)^2$$

Equation 4 Mean Squared Error (Schott, 2019)

Where:

y_i = actual value

f_i = predicted value at that node

n = number of samples

A lower MSE indicates that the node is more "pure," making the split more favorable. Random Forest regression combines the predictions from all trees, which decreases variance and enhances stability.

B. Classification Issues with balanced dataset (utilizing Gini Index)

For classification tasks utilizing balanced dataset, Random Forest usually assesses the quality of splits by measuring Gini Impurity:

$$Gini = 1 - \sum_{i=1}^c (P_i)^2$$

Equation 5 Gini Impurity (Schott, 2019)

Where:

p_i = probability (relative frequency) of class i at the node

c = total number of classes

A smaller Gini value signifies a purer node. Random Forest chooses the split that leads to the maximum decrease in Gini impurity.

C. Classification Issues with imbalanced dataset (utilizing Entropy (Information Gain))

For classification tasks utilizing imbalanced dataset, Random Forest usually assesses the quality of splits by measuring Entropy:

$$Entropy = \sum_{i=1}^c -p_i * \log_2(p_i)$$

Equation 6 Entropy (Schott, 2019)

Where:

p_i = probability (relative frequency) of class i at the node

c = total number of classes

Entropy quantifies the uncertainty at a node. A split that minimizes entropy the most is selected. While entropy is a bit more computationally intensive than Gini because of the logarithm, both methods typically yield similar outcomes (Schott, 2019).

3.1.3 XGBoost

XGBoost is an efficient version of Gradient Boosting and is a form of ensemble learning that merges several weak models to create a more powerful model (Anon., 2025). XGBoost employs decision trees as its foundational learners and adds them one after another to enhance the model's effectiveness (XGBoost, 2025). Each new tree is designed to fix the mistakes of the prior tree, a method known as boosting (XGBoost, 2025). It has integrated parallel processing to rapidly train models on extensive datasets. Additionally, XGBoost allows for customizations, enabling users to modify model parameters to enhance performance tailored to the specific issue (XGBoost, 2025).



Figure 6 XGBoost (Anon., 2025)

Mathematically, it can be seen as an iterative process that begins with an initial prediction, usually set to zero. Then, each tree is added to minimize errors.

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

Equation 7 XGBoost Model (XGBoost, 2025)

Where:

- $\hat{y}_i$ is the final predicted value for the i_{th} data point
- K is the number of trees in the ensemble
- $f_k(x_i)$ represents the prediction of the K_{th} tree for the i_{th} data point (XGBoost, 2025)

The objective function in XGBoost has two components: a loss function and a regularization term. The loss function evaluates how accurately the model represents the data, while the regularization term helps to simplify complex trees (XGBoost, 2025). The typical structure of the loss function is:

$$obj(\theta) = \sum_i^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Equation 8 objective function in XGBoost (XGBoost, 2025)

Where:

- $l(y_i, \hat{y}_i)$ is the loss function that calculates the difference between the actual value y_i and the predicted value $\hat{y}_i$
- $\Omega(f_k)$ is the regularization term that prevents trees from becoming too complex (XGBoost, 2025)

Now, rather than fitting the model in one go, we optimize it step by step. We begin with an initial prediction $\hat{y}_i(0) = 0$ and at every step, we introduce a new tree to enhance the model (XGBoost, 2025). The revised predictions after incorporating the t_{th} tree can be expressed as:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_x(x_i)$$

Equation 9 15 Additive Training (Boosting Framework) (XGBoost, 2025)

Where:

- $\hat{y}_i^{(t-1)}$ refers to the previous iteration prediction
- $f_t(x_i)$ is the t_{th} tree prediction for the i_{th} data point (XGBoost, 2025)

Ultimately, when determining how to divide the nodes in the tree, we calculate the information gain for each potential split (XGBoost, 2025). The information gain for a split is determined as:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} + \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

Equation 10 Information gain (XGBoost, 2025)

Where:

G_L , G_R are the total gradients from the left and right child nodes

H_L , H_R are the total Hessians from the left and right child nodes (XGBoost, 2025)

XGBoost determines the best split by calculating the information gain for all potential splits at each node, choosing the one that maximizes the gain, thereby minimizing errors and enhancing the model's performance (XGBoost, 2025).

3.2 PSEUDOCODE

3.2.1 Pseudocode for Logistic Regression

BEGIN

Initialize weights w and bias b to small values

Set learning rate α

FOR each epoch **DO**

FOR each training sample (x_i, y_i) **DO**

 Compute linear score: $z_i = w \cdot x_i + b$

 Compute predicted probability: $p_i = 1 / (1 + e^{(-z_i)})$

 Compute error: $e_i = p_i - y_i$

UPDATE weights: $w = w - \alpha * (e_i * x_i)$

UPDATE bias: $b = b - \alpha * e_i$

END FOR

END FOR

RETURN trained weights w and bias b

END

3.2.2 Pseudocode for Random Forest

BEGIN

INPUT: training dataset D , number of trees T

FOR $t = 1$ **TO** T **DO**

 Draw bootstrap sample D_t from D

 Initialize decision tree $Tree_t$

WHILE tree can be split **DO**

 Randomly select subset of features F

FOR each possible feature f in F **DO**

 Evaluate all possible split points for f

 Compute impurity reduction (e.g., Gini or Entropy)

END FOR

 Select split with maximum impurity reduction

 Partition node into left and right child nodes

END WHILE

 Add trained $Tree_t$ to the forest

END FOR

RETURN all trees forming the Random Forest model

END

3.2.3 Pseudocode for XGBoost

BEGIN

INPUT: training data D , number of boosting rounds K

Initialize predictions $\hat{y}_i = 0$ for all samples

FOR $k = 1$ **TO** K **DO**

FOR each sample i **DO**

 Compute gradient: $g_i = \partial L(y_i, \hat{y}_i) / \partial \hat{y}_i$

 Compute hessian: $h_i = \partial^2 L(y_i, \hat{y}_i) / \partial \hat{y}_i^2$

END FOR

Build tree f_k by minimizing:

 Objective = $\sum (g_i * w_j + 1/2 * h_i * w_j^2) + \text{Regularization}$

FOR each leaf j **DO**

 Compute optimal leaf weight:

$w_j = - (\sum g_i \text{ in leaf } j) / (\sum h_i \text{ in leaf } j + \lambda)$

END FOR

Update predictions:

$\hat{y}_i = \hat{y}_i + \eta * f_k(x_i)$

END FOR

RETURN all trees $f_1, f_2, \dots, f_k$

END

3.3. Diagrammatical representations of the solution

3.3.1 Proposed Model Diagram

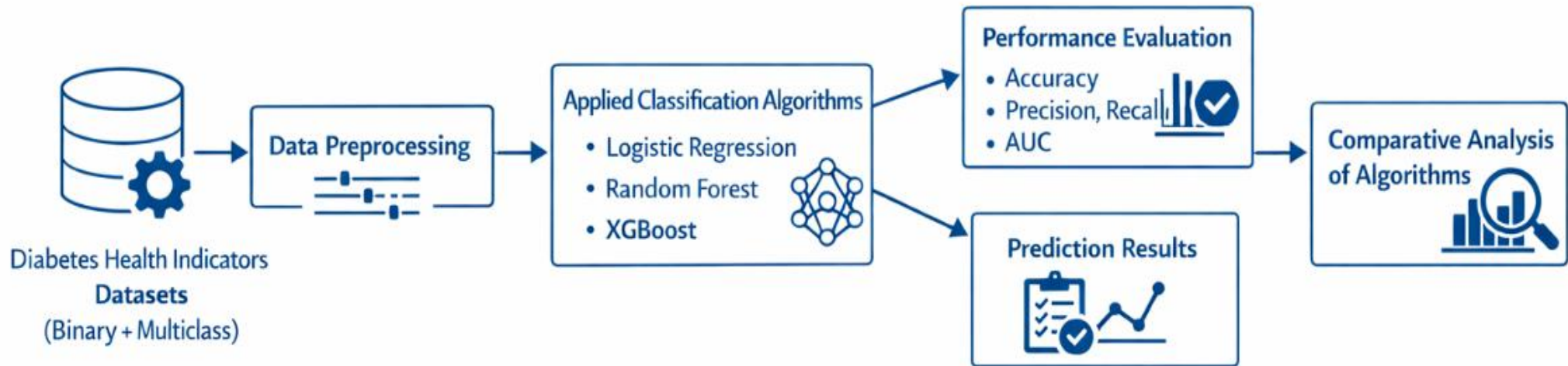


Figure 7 Proposed Model Diagram

3.3.2 Flowcharts

3.3.2.1 System Architecture Flowchart

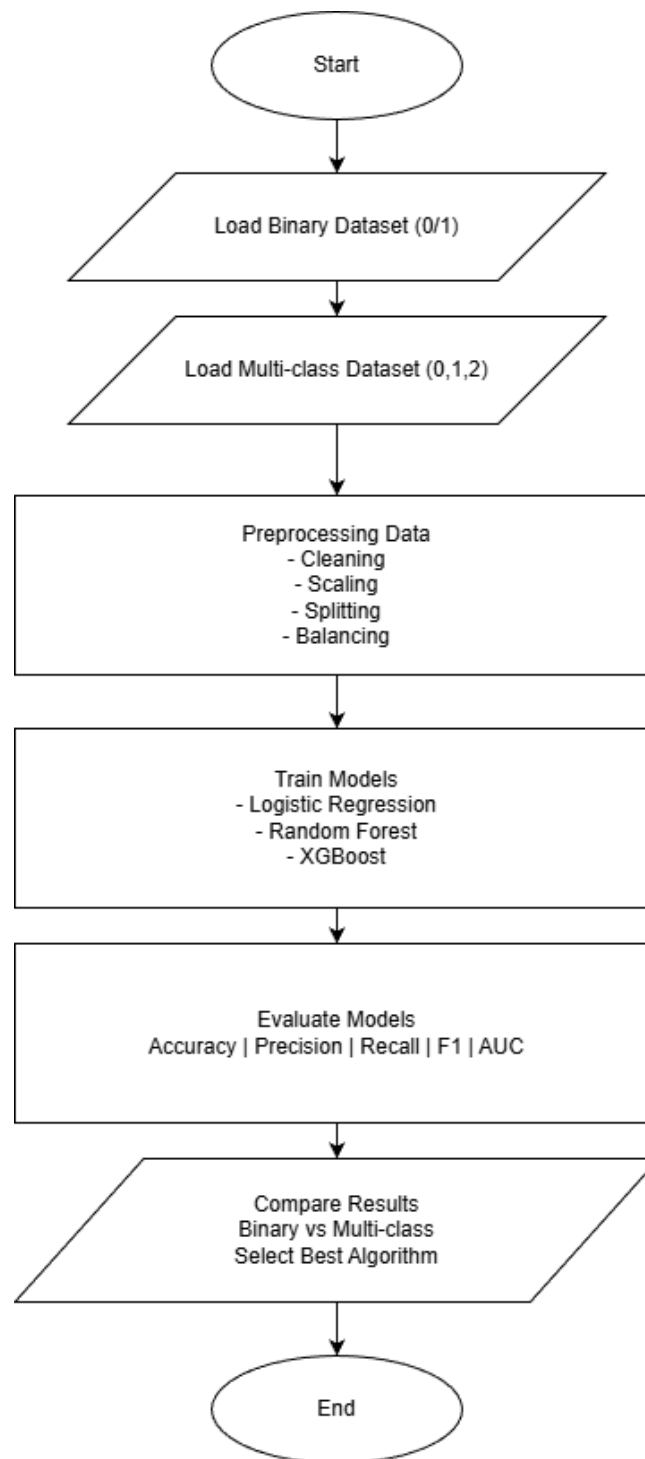


Figure 8 System Architecture Flowchart

3.3.2.2 Logistic Regression Flowchart

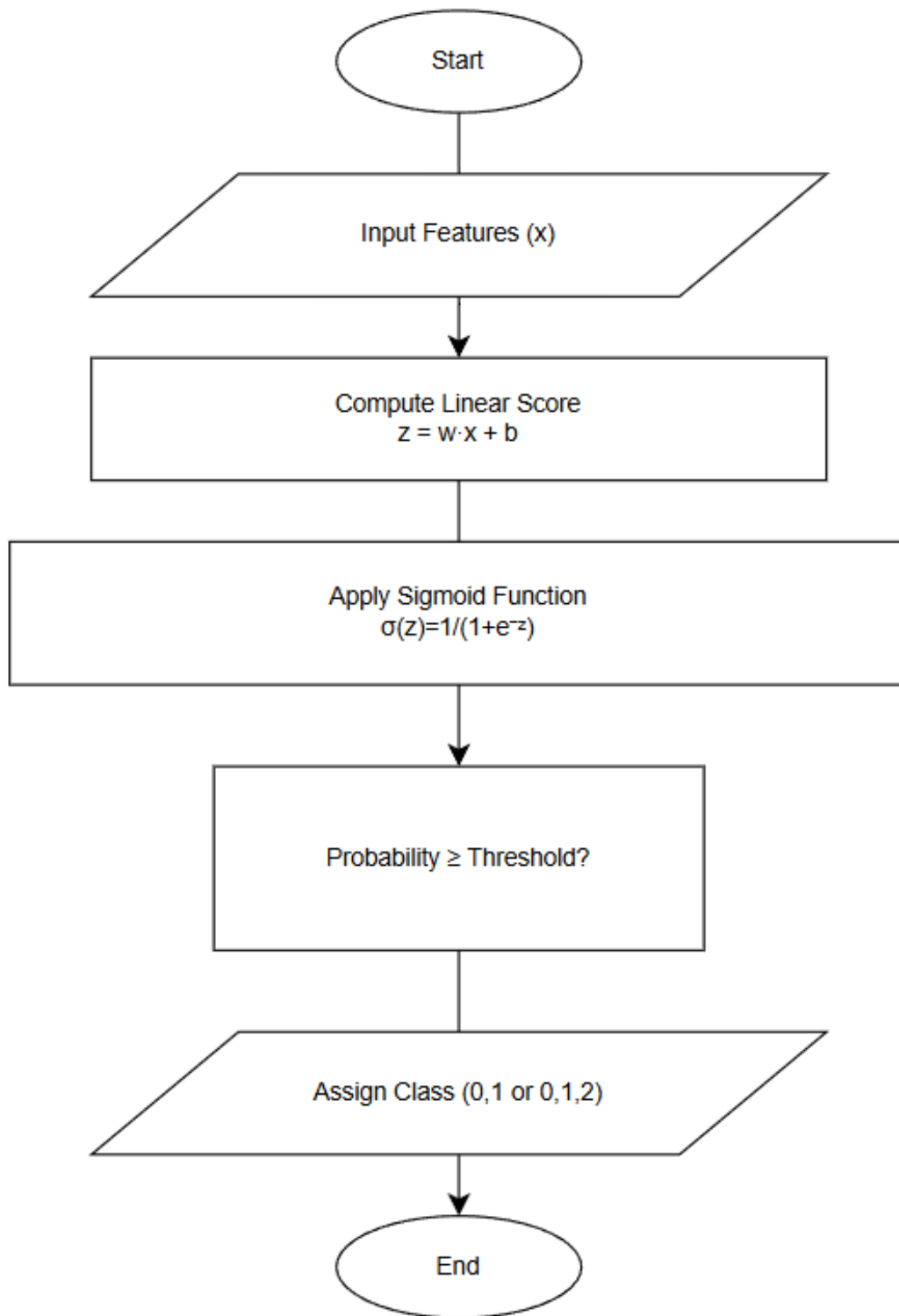


Figure 9 Logistic Regression Flowchart

3.3.2.3 Random Forest Flowchart

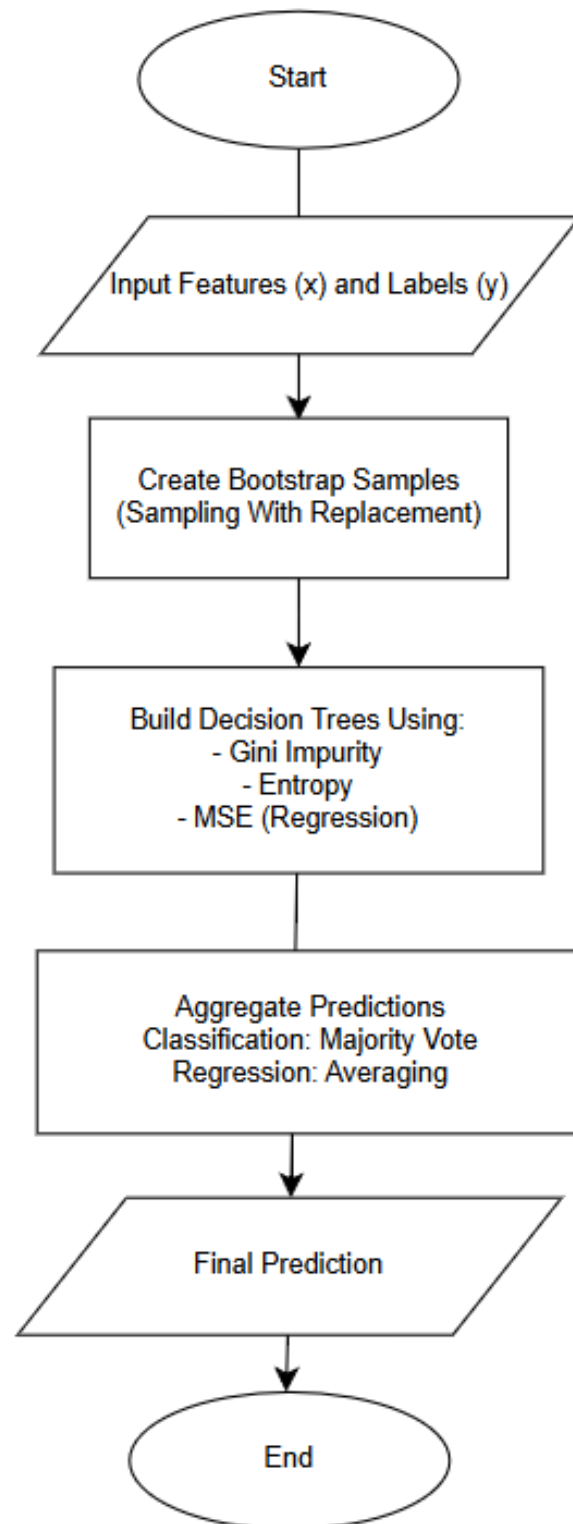


Figure 10 Random Forest Flowchart

3.3.2.4 XGBoost Flowchart

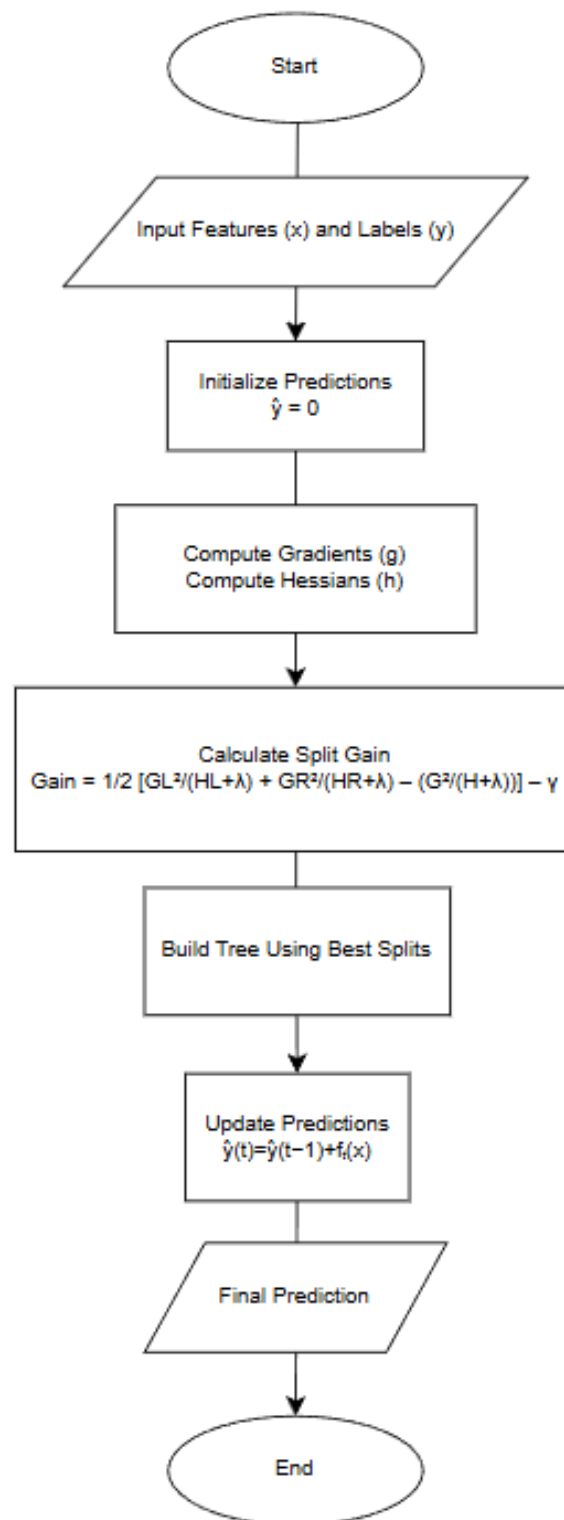


Figure 11 XGBoost Flowchart

3.3.3 State Transition Diagram

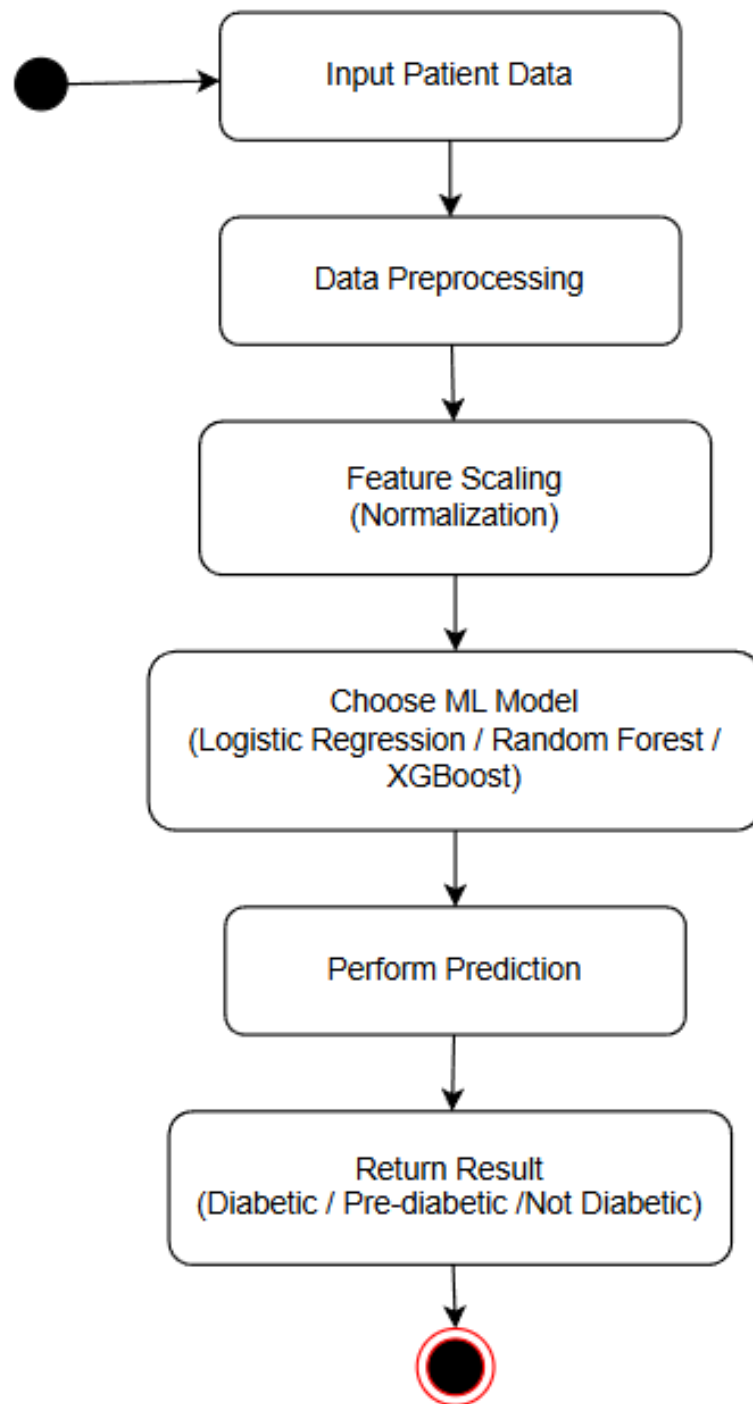


Figure 12 State Transition Diagram

3.4 Tools and Technologies Used

This project used a mix of open-source programming languages, libraries, and computational tools to assist with data preprocessing, exploratory analysis, model development, evaluation, and visualization. The chosen technologies were selected for their strength, reproducibility, and common use in machine learning and healthcare analytics research.

3.4.1 Programming Language

1. Python



Figure 13 Python

Python was chosen as the main programming language for this project because of its vast array of data science and machine learning libraries, its readability, and the robust support from its community (Duda, 2021). Python allows for quick prototyping and smooth integration of data preprocessing, modeling, and evaluation processes all in one environment (Duda, 2021)

Python was utilized to:

- Load and handle large healthcare datasets
- Carry out exploratory data analysis (EDA)
- Apply machine learning algorithms
- Perform cross-validation and assessment
- Create high-quality visualizations suitable for publication

3.4.2 Data Management and Numerical Computing

1. Pandas



Figure 14 Pandas

Pandas was used for loading, cleaning, transforming, and exploring data. Its DataFrame format allows for effective management of structured tabular data and made tasks like filtering, aggregation, and feature selection easier (Mckinney, 2011).

Pandas was utilized to:

- Load CSV files
- Check data types and identify missing values
- Analyze class distribution
- Prepare features and target variables for modeling

2. NumPy



Figure 15 NumPy

NumPy was used for numerical calculations and efficient array operations. It offered essential support for mathematical transformations, calculating class weights, and working with machine learning libraries (Perepu, 2025)

NumPy was utilized to:

- Carry out vectorized numerical operations

- Calculate class weights
- Assist with fundamental mathematical computations

3.4.3 Data Visualization

1. Matplotlib



Figure 16 Matplotlib

Matplotlib served as the main plotting library for creating static and customizable visualizations as it allowed detailed control over plot components and was widely used for performance comparison and evaluation plots (Barrett, 2005)

Matplotlib was utilized to:

- Generate bar charts and line plots
- Visualize ROC curves
- Plot learning curves and performance comparisons

2. Seaborn



Figure 17 Seaborn

Seaborn was used to produce high-level statistical visualizations based on Matplotlib. It was especially beneficial for exploratory data analysis and visualizing relationships.

Seaborn was employed to:

- Create correlation heatmaps
- Visualize class distributions
- Plot boxplots and histograms for outlier analysis

3.4.3 Machine Learning Frameworks

1. Scikit-learn



Figure 18 Scikit learn

Scikit-learn was the main machine learning framework for this project. It offers dependable implementations of traditional machine learning algorithms and standardized tools for evaluation and validation (Pedregosa, 2012)

Scikit-learn was utilized to:

- Create Logistic Regression and Random Forest models
- Conduct train–test splitting
- Use stratified k-fold cross-validation
- Calculate evaluation metrics like precision, recall, F1-score, and ROC-AUC

2. Imbalanced-learn (SMOTE)



Figure 19 Imbalanced-learn

The Imbalanced-learn library was employed to tackle significant class imbalance in both binary and multiclass datasets, in particular, the Synthetic Minority Over-sampling Technique (SMOTE) was used to create synthetic samples for minority classes (Chawla, 2002)

Imbalanced-learn was applied to:

- Equalize class distributions in training data

- Enhance model learning for minority classes such as prediabetes
- Avoid model bias towards majority classes

3. XGBoost



Figure 20 XGBoost

XGBoost was used as a sophisticated ensemble learning algorithm because of its capability to capture complex non-linear relationships and its excellent performance on structured tabular data (Chen, 2016)

XGBoost was utilized to:

- Train gradient-boosted decision tree models
- Manage class imbalance using `scale_pos_weight`
- Enhance predictive performance through boosting techniques

3.4.3 Development Environment

1. Google Colab



Figure 21 Google Colab

This project utilized Google Colab as its development environment. It offers cloud-based computing resources, removes the need for local setup, and facilitates collaborative experimentation (Pessoa, 2018). Google Colab was employed to:

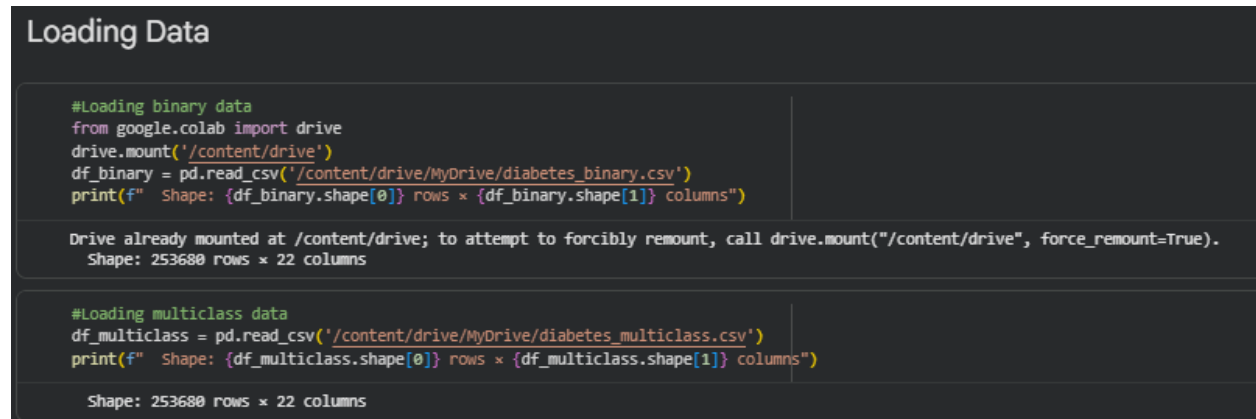
- Run computationally heavy models
- Save and retrieve datasets through Google Drive
- Guarantee the reproducibility of experiments

3.5 Development Process

3.5.1 Data Loading

The project utilized the CDC BRFSS 2015 dataset, which includes the following details:

- Total samples: 253,680 complete records
- Features: 21 health indicators
- Feature types: 14 binary features, 3 continuous features and 4 ordinal features
- Binary target: 218,334 (86.1%) with No Diabetes, 35,346 (13.9%) with Diabetes
- Multiclass target: 213,703 (84.2%) with No Diabetes, 4,631 (1.8%) with Prediabetes, and 35,346 (13.9%) with Diabetes



The screenshot shows a Jupyter Notebook interface with a dark theme. The title bar at the top says "Loading Data". The code cell contains two sections of Python code. The first section, "#Loading binary data", imports the 'drive' module from 'google.colab', mounts the '/content/drive' directory, reads a CSV file 'diabetes_binary.csv' into a DataFrame 'df_binary', and prints its shape. The output shows the drive is already mounted and the DataFrame has 253680 rows and 22 columns. The second section, "#Loading multiclass data", reads a CSV file 'diabetes_multiclass.csv' into a DataFrame 'df_multiclass' and prints its shape. The output shows the DataFrame has 253680 rows and 22 columns.

```
#Loading binary data
from google.colab import drive
drive.mount('/content/drive')
df_binary = pd.read_csv('/content/drive/MyDrive/diabetes_binary.csv')
print(f" Shape: {df_binary.shape[0]} rows x {df_binary.shape[1]} columns")

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Shape: 253680 rows x 22 columns

#Loading multiclass data
df_multiclass = pd.read_csv('/content/drive/MyDrive/diabetes_multiclass.csv')
print(f" Shape: {df_multiclass.shape[0]} rows x {df_multiclass.shape[1]} columns")

Shape: 253680 rows x 22 columns
```

Figure 22 Data Loading

3.5.2. Data Cleaning

After looking closely at both the binary and multiclass datasets, it was found that there were no missing values. Still, to keep everything accurate, it was made sure to spot and eliminate any duplicate entries.

```
missing_binary = df_binary.isnull().sum()
print(missing_binary)

df_binary.drop_duplicates(subset=None, keep='first', inplace=False, ignore_index=False)
```

BINARY DATASET OVERVIEW

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 253680 entries, 0 to 253679
Data columns (total 22 columns):

#	Column	Non-Null Count	Dtype
0	Diabetes_binary	253680 non-null	float64
1	HighBP	253680 non-null	float64
2	HighChol	253680 non-null	float64
3	CholCheck	253680 non-null	float64
4	BMI	253680 non-null	float64
5	Smoker	253680 non-null	float64
6	Stroke	253680 non-null	float64
7	HeartDiseaseorAttack	253680 non-null	float64
8	PhysActivity	253680 non-null	float64
9	Fruits	253680 non-null	float64
10	Veggies	253680 non-null	float64
11	HvyAlcoholConsump	253680 non-null	float64
12	AnyHealthcare	253680 non-null	float64
13	NoDocbcCost	253680 non-null	float64
14	GenHlth	253680 non-null	float64
15	MentHlth	253680 non-null	float64
16	PhysHlth	253680 non-null	float64
17	DiffWalk	253680 non-null	float64
18	Sex	253680 non-null	float64
19	Age	253680 non-null	float64
20	Education	253680 non-null	float64
21	Income	253680 non-null	float64

dtypes: float64(22)
memory usage: 42.6 MB
None

Missing Values:

Diabetes_binary	0
HighBP	0
HighChol	0
CholCheck	0
BMI	0
Smoker	0
Stroke	0
HeartDiseaseorAttack	0
PhysActivity	0
Fruits	0
Veggies	0
HvyAlcoholConsump	0

Figure 23 Data Cleaning

3.5.3. Exploratory Data Analysis

i) Class Distribution Analysis

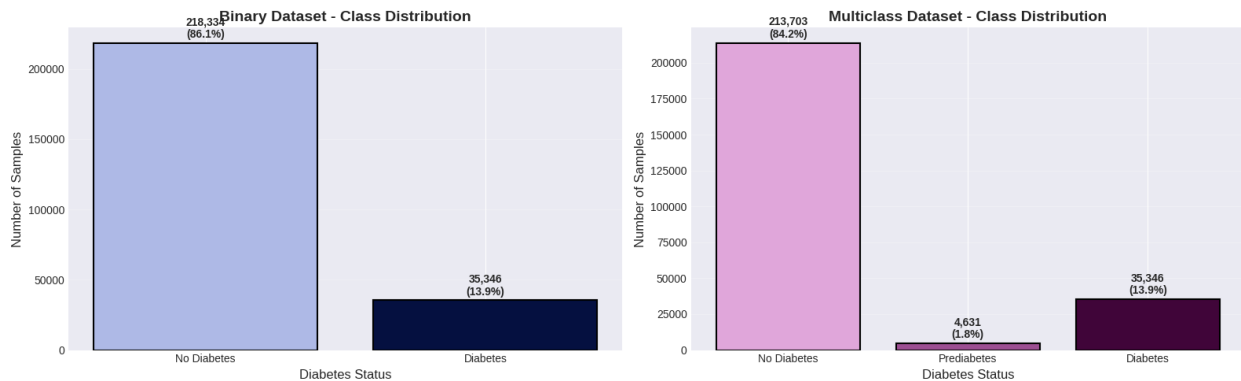


Figure 24 Class Distribution Analysis

Grasping the distribution of the target variable was essential before diving into model development, as class imbalance can lead to models that favor the majority class. The bar chart visualizations clearly showed that the non-diabetic class was predominant in both the datasets, while the minority classes, especially Prediabetes, were significantly underrepresented. To tackle this imbalance and ensure our model performs well, SMOTE (Synthetic Minority Oversampling Technique) was implemented on both datasets. This approach helped create balanced training sets, allowing for fair learning across all classes.

ii) Correlation Analysis

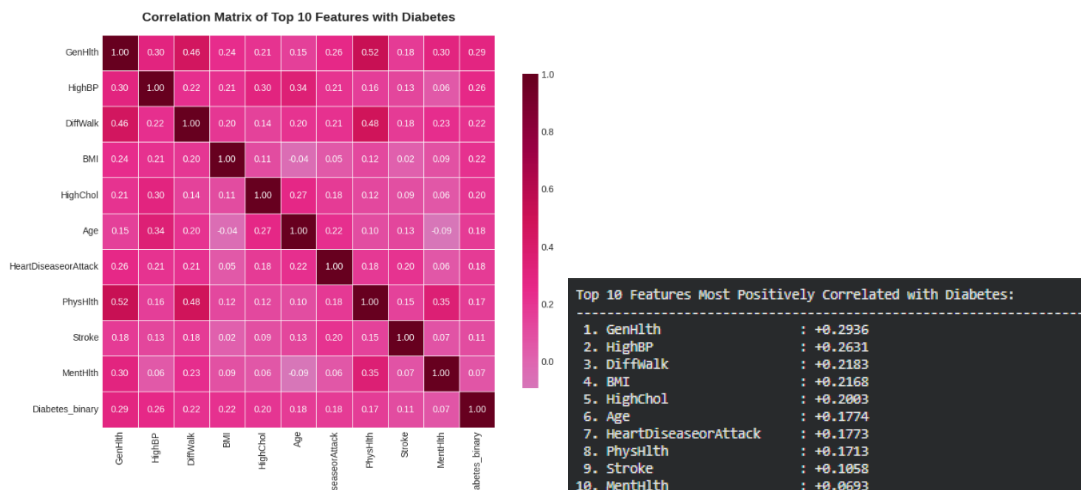


Figure 25 Correlation Heatmap for features

A correlation analysis was conducted on a binary dataset to pinpoint the features that are most closely linked to diabetes. By calculating a correlation matrix for all the numerical features against the target variable (Diabetes_binary), the features were ranked based on their correlation strength. The top 10 predictors included BMI, General Health (GenHlth), Physical Health (PhysHlth), Mental Health (MentHlth), and Age, with correlation coefficients that varied from moderately strong to weak. These results indicate that BMI, GenHlth, PhysHlth, and Age are likely to play a significant role in model predictions, while the features with weaker correlations might not add much to the predictive power.

iii) Outlier Detection

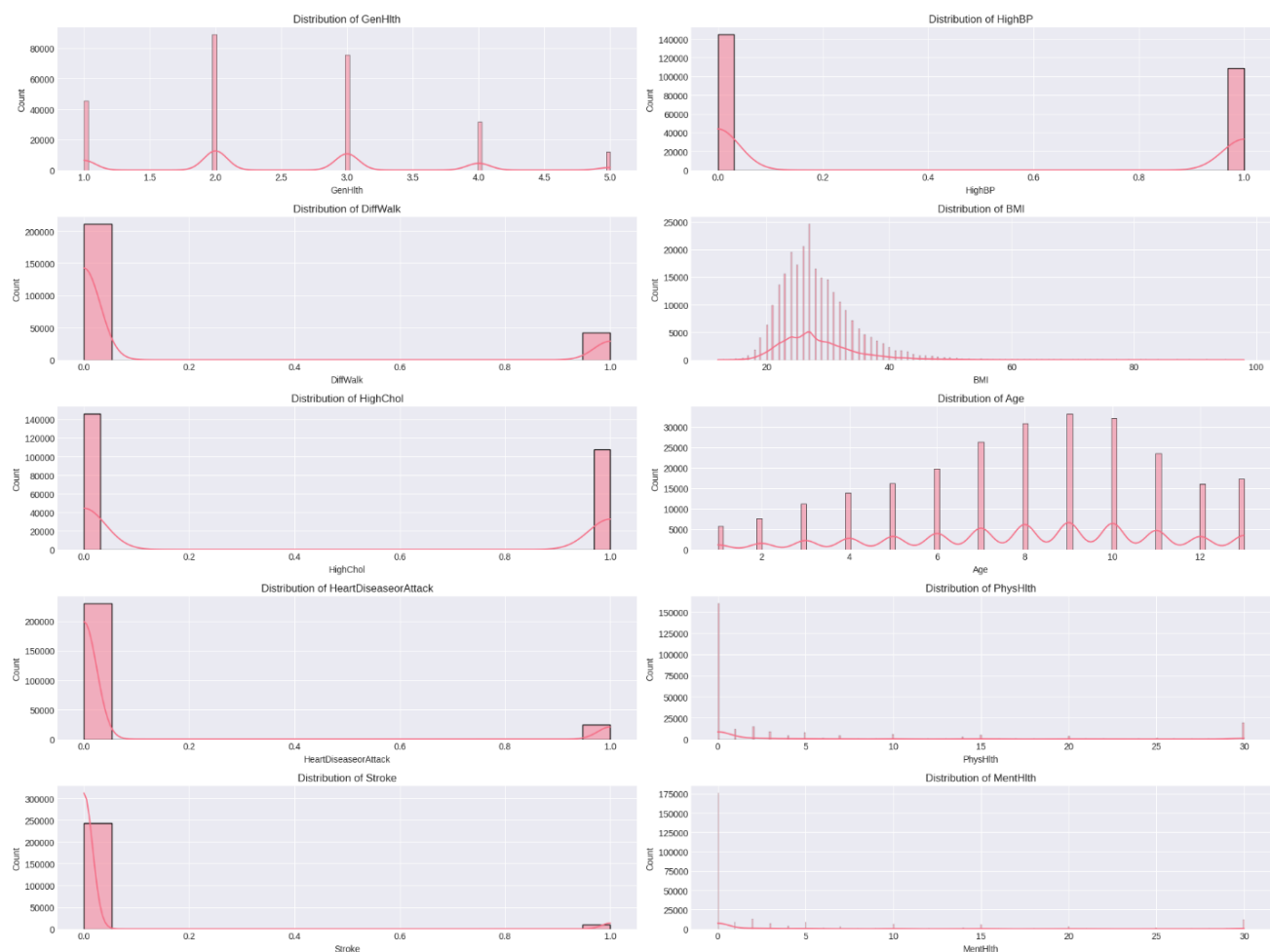


Figure 26 Outlier Detection Histplots

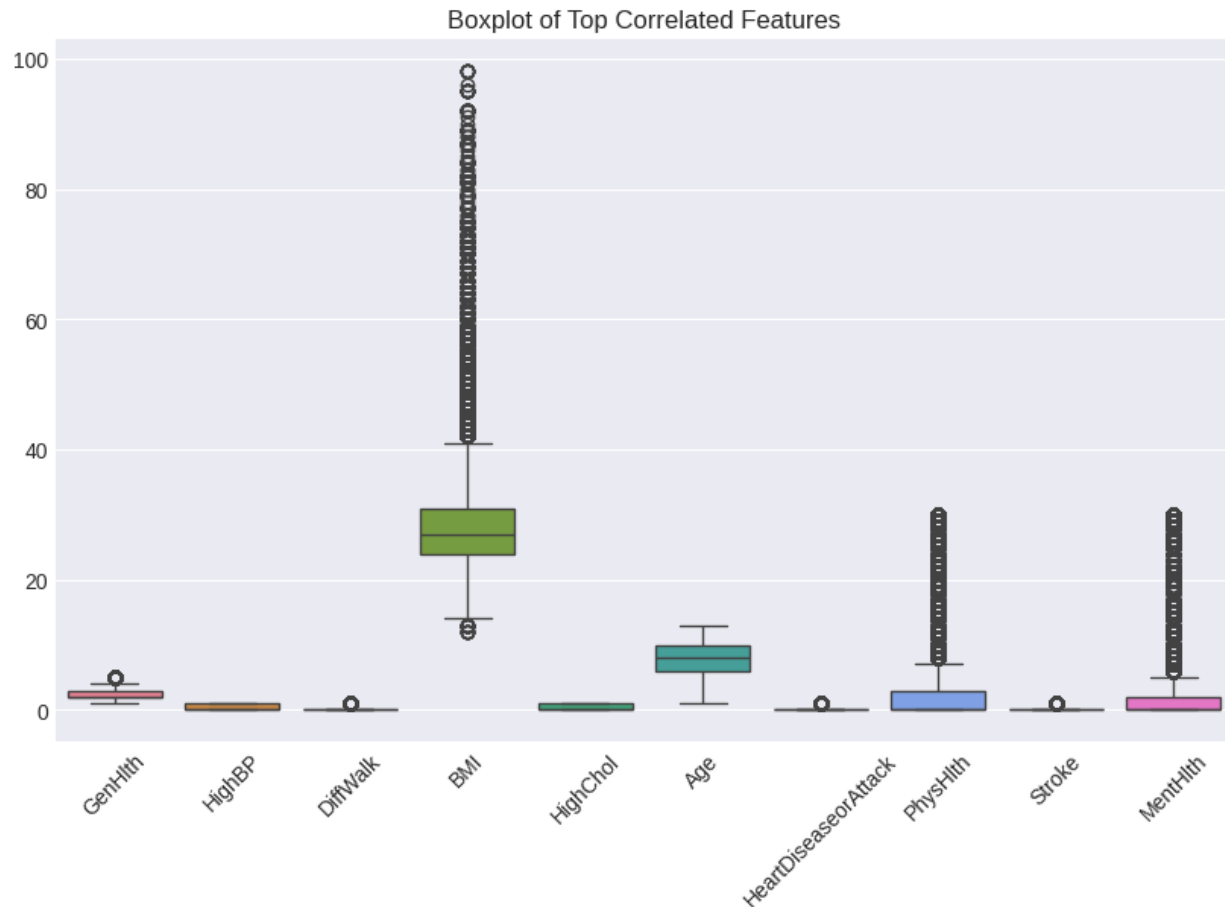


Figure 27 Box plots

After conducting correlation analysis, outlier detection was performed to investigate feature distributions and pinpoint extreme values that could affect model performance.

Boxplots illustrated the median, quartiles, and possible outliers for the most positively correlated features, while histograms with kernel density estimates (KDE) displayed overall distribution trends and potential skewness.

Both techniques revealed a limited number of extreme values in features like BMI, MentHlth, and PhysHlth. Nevertheless, histograms indicated that the majority of observations were clustered within standard ranges, with outliers being infrequent. These outliers seemed to represent legitimate, though atypical, cases instead of data inaccuracies. As a result, due to their low occurrence and the intended use of feature scaling, no records were discarded. Feature scaling will help reduce any disproportionate impact of extreme values on model performance

3.5.4. Data Pre-processing

i) Data Scaling

To ensure equal contribution of numerical features to model training, scaling was applied to continuous variables. Features like BMI, MentHlth, and PhysHlth underwent z-score normalization. This method adjusts each feature to have a mean of 0 and a standard deviation of 1, minimizing the effects of varying ranges among features.

Procedure:

- Continuous features were initially identified through domain knowledge and data exploration.
- StandardScaler from scikit-learn was utilized to fit and transform the training set.
- The same scaler parameters were subsequently later applied to the testing set to ensure consistency.

```
# Continuous features that need scaling
continuous_features = ['BMI', 'MentHlth', 'PhysHlth']
print("Before Scaling - Feature Ranges:")
print("-"*80)
for feature in continuous_features:
    print(f"{feature:12s}: Min={X_binary[feature].min():.2f}, Max={X_binary[feature].max():.2f}, "
          f"Mean={X_binary[feature].mean():.2f}, Std={X_binary[feature].std():.2f}")

#Scaling the feature
scaler = StandardScaler()
X_binary_scaled = X_binary.copy()
X_multiclass_scaled = X_multiclass.copy()
X_binary_scaled[continuous_features] = scaler.fit_transform(X_binary[continuous_features])
X_multiclass_scaled[continuous_features] = scaler.transform(X_multiclass[continuous_features])
```

Figure 28 Data Scaling Procedure

Results:

- Before scaling, features displayed different ranges; for instance, BMI varied from approximately 12 to 98, while MentHlth and PhysHlth ranged from 0 to 30.
- Post-scaling, all continuous features were normalized to a unified scale, with a mean close to 0 and a standard deviation of 1.
- This standardization mitigates potential bias during model training and enhances the convergence of gradient-based algorithms.

```

Before Scaling - Feature Ranges:
-----
BMI      : Min=12.00, Max=98.00, Mean=28.38, Std=6.61
MentHlth : Min=0.00, Max=30.00, Mean=3.18, Std=7.41
PhysHlth : Min=0.00, Max=30.00, Mean=4.24, Std=8.72
After Scaling - Feature Ranges:
-----
BMI      : Min=-2.48, Max=10.53, Mean=-0.00, Std=1.00
MentHlth : Min=-0.43, Max=3.62, Mean=0.00, Std=1.00
PhysHlth : Min=-0.49, Max=2.95, Mean=0.00, Std=1.00

```

Figure 29 Data Scaling Result

ii) Train Test Split

Following scaling, the datasets were divided into training and testing subsets to assess model performance on new data. A stratified split was utilized to maintain the original class distributions in both binary and multiclass datasets.

Procedure:

- The binary dataset was divided with 80% allocated for training and 20% for testing.
- The multiclass dataset was divided in a similar manner, ensuring that each class retained its proportion in both subsets.

```

# Binary Dataset Split
X_train_binary, X_test_binary, y_train_binary, y_test_binary = train_test_split(X_binary_scaled, y_binary, test_size=0.2)
print("BINARY DATASET SPLIT:")
print("-"*80)
print(f"Training set: {X_train_binary.shape[0]:,} samples ({X_train_binary.shape[0]/len(X_binary_scaled)*100:.1f}%")
print(f"Testing set: {X_test_binary.shape[0]:,} samples ({X_test_binary.shape[0]/len(X_binary_scaled)*100:.1f}%")

# Multiclass Dataset Split
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(X_multiclass_scaled, y_multiclass, test_size=0.2)
print("MULTICLASS DATASET SPLIT:")
print("-"*80)
print(f"Training set: {X_train_multi.shape[0]:,} samples ({X_train_multi.shape[0]/len(X_multiclass_scaled)*100:.1f}%")
print(f"Testing set: {X_test_multi.shape[0]:,} samples ({X_test_multi.shape[0]/len(X_multiclass_scaled)*100:.1f}%")

BINARY DATASET SPLIT:
-----
Training set: 202,944 samples (80.0%)
Testing set: 50,736 samples (20.0%)
MULTICLASS DATASET SPLIT:
-----
Training set: 202,944 samples (80.0%)
Testing set: 50,736 samples (20.0%)

```

Figure 30 Train Test Split

iii) Class Balancing

To tackle class imbalance and enhance model performance for minority classes, SMOTE (Synthetic Minority Oversampling Technique) was utilized:

- In the binary dataset, the minority class (Diabetes) was oversampled to align with the majority class (No Diabetes).
- In the multiclass dataset, the Prediabetes and Diabetes classes were oversampled to correspond with the No Diabetes class.

```
# Binary Dataset SMOTE
train_dist_binary = pd.Series(y_train_binary).value_counts().sort_index()
smote_binary = SMOTE(sampling_strategy=1.0, random_state=42)
X_train_binary_smote, y_train_binary_smote = smote_binary.fit_resample(X_train_binary, y_train_binary)
smote_dist_binary = pd.Series(y_train_binary_smote).value_counts().sort_index()

print(f"\nBefore SMOTE:")
print(f" No Diabetes: {train_dist_binary[0]:,}")
print(f" Diabetes:    {train_dist_binary[1]:,}")
print(f" Total:       {len(y_train_binary):,}")

print(f"\nAfter SMOTE:")
print(f" No Diabetes: {smote_dist_binary[0]:,}")
print(f" Diabetes:    {smote_dist_binary[1]:,}")
print(f" Total:       {len(y_train_binary_smote):,}")

# Multiclass Dataset SMOTE
train_dist_multi = pd.Series(y_train_multi).value_counts().sort_index()
majority_count = train_dist_multi[0]
sampling_strategy_multi = {0: majority_count, 1: majority_count, 2: majority_count}
smote_multi = SMOTE(sampling_strategy=sampling_strategy_multi, random_state=42, k_neighbors=5)
X_train_multi_smote, y_train_multi_smote = smote_multi.fit_resample(X_train_multi, y_train_multi)
smote_dist_multi = pd.Series(y_train_multi_smote).value_counts().sort_index()
```

Figure 31 Class Balancing SMOTE procedure

Results:

- After applying SMOTE, both datasets exhibited balanced class distributions.
- This balancing prevents models from developing a bias towards majority classes and allows for the learning of significant patterns across all target classes.

Before SMOTE:	Before SMOTE:
No Diabetes: 174,667	Class 0.0: 170,962 (84.24%)
Diabetes: 28,277	Class 1.0: 3,705 (1.83%)
Total: 202,944	Class 2.0: 28,277 (13.93%)
	Total: 202,944
After SMOTE:	After SMOTE:
No Diabetes: 174,667	Class 0.0: 170,962 (33.33%)
Diabetes: 174,667	Class 1.0: 170,962 (33.33%)
Total: 349,334	Class 2.0: 170,962 (33.33%)
	Total: 512,886

Figure 32 Class Balancing SMOTE result

Visualizations:

Class distributions before and after SMOTE were illustrated, effectively demonstrating the rectification of imbalance in both binary and multiclass datasets.

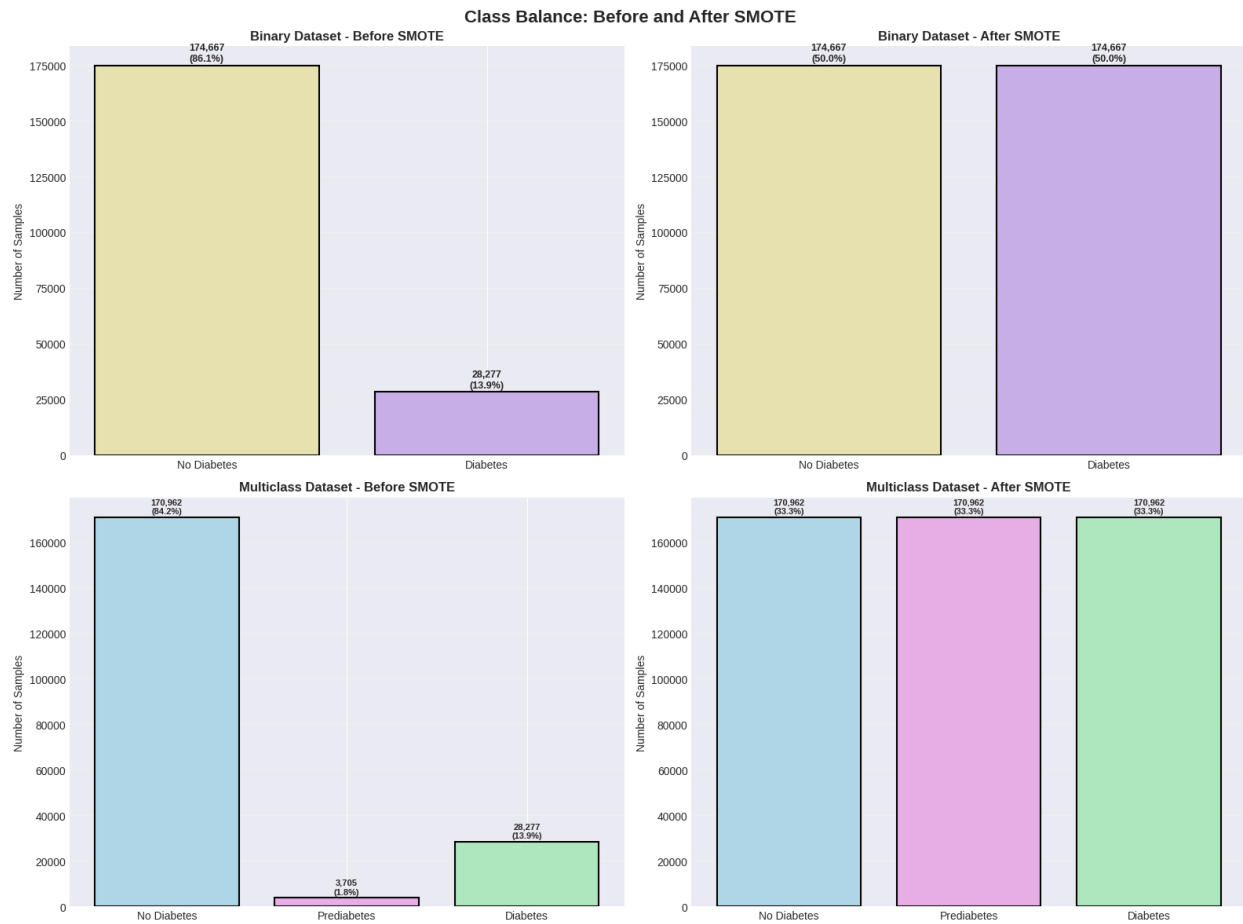


Figure 33 Class Balancing SMOTE visualizations

3.5.5. Model Definition with class weights

After addressing class imbalance using SMOTE, three classification algorithms were established for both binary and multiclass diabetes prediction tasks: Logistic Regression, Random Forest, and XGBoost. To enhance model performance on minority classes, class weights were calculated and utilized for Logistic Regression and Random Forest.

Binary Classification:

- Class weights were determined using `compute_class_weight` from `sklearn` based on the resampled SMOTE training dataset.

```
# Binary class weights
class_weights_binary = dict(zip(
    np.unique(y_train_binary_smote),
    compute_class_weight(class_weight='balanced', classes=np.unique(y_train_binary_smote), y=y_train_binary_smote)
))
```

Figure 34 Defining class weights

- Logistic Regression and Random Forest models applied these class weights to penalize the misclassification of the minority class (diabetes).

```
models_binary = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42, class_weight=class_weights_binary),
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=20, random_state=42, n_jobs=-1, class_weight=class_weights_binary),
    'XGBoost': xgb.XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, objective='binary:logistic',
    eval_metric='logloss', use_label_encoder=False, random_state=42, n_jobs=-1, scale_pos_weight=y_train_binary_smote.value_counts()[0] / y_
}
```

Figure 35 Binary model definition

- For XGBoost, `scale_pos_weight` was computed as the ratio of majority to minority samples, enabling the model to better manage the imbalance while maintaining the SMOTE-adjusted distribution.

Multiclass Classification:

- Class weights were similarly calculated for the three classes (No Diabetes, Prediabetes, Diabetes) using the SMOTE-resampled training dataset.

```
# Multiclass class weights
class_weights_multi = dict(zip(
    np.unique(y_train_multi_smote),
    compute_class_weight(class_weight='balanced', classes=np.unique(y_train_multi_smote), y=y_train_multi_smote)
))
```

Figure 36 Defining class weights multiclass

- Logistic Regression and Random Forest models integrated these class weights to emphasize underrepresented classes.

```
# Multiclass Classification Models
models_multi = {
    'Logistic Regression': LogisticRegression(max_iter=1000,multi_class='multinomial',random_state=42,class_weight=class_weights_multi),
    'Random Forest': RandomForestClassifier(n_estimators=200,max_depth=20,random_state=42,n_jobs=-1,class_weight=class_weights_multi),
    'XGBoost': xgb.XGBClassifier(n_estimators=200,max_depth=6,learning_rate=0.1,subsample=0.8,colsample_bytree=0.8,objective='multi:softprob',eval_metric='mlogloss',use_label_encoder=False,random_state=42,n_jobs=-1)
}
```

Figure 37 Multiclass model definition

- XGBoost did not need explicit class weights for multiclass since SMOTE already balanced the classes throughout the training set.

Hyperparameters adjustment table

Attempt	Models Used	Key Configuration / Changes	Reason / Observations	Outcome / Metrics / Notes
1	Logistic Regression, Random Forest, XGBoost	Trained on SMOTE-balanced data; default hyperparameters	Baseline after class balancing	Binary F1 for minority ~0.3; Multiclass prediabetic F1 ~0.01; poor minority class performance.
2	Logistic Regression, Random Forest, XGBoost	Added tuned hyperparameters: n_estimators, max_depth, learning_rate	To improve model capacity and generalization	Slight improvement, but minority classes still predicted poorly; confusion matrices showed skewed predictions.
3	Logistic Regression & Random Forest	Added class_weight='balanced'	Give more importance to minority classes	Minor increase in recall for minority class; still low F1 for prediabetic.

Attempt	Models Used	Key Configuration / Changes	Reason / Observations	Outcome / Metrics / Notes
4	Binary XGBoost	Added scale_pos_weight= int(major/minor)	Weighted binary classes to address imbalance	Improved diabetic class recall slightly; integer weighting not precise enough.
5	Cross- validation (all models)	5-fold StratifiedKFold; scoring: F1 / F1- weighted	Fair evaluation across models	Provided reliable comparison of models; revealed LR underperforms for minority.
6	Binary XGBoost	Tuned scale_pos_weight using float division (major/minor)	More precise class weighting	F1 and recall for diabetic class improved; precision slightly decreased but acceptable.
7	Binary & Multiclass	Threshold tuning post-prediction (e.g., 0.3 for minority)	Capture more minority class samples	Recall for minority classes improved; precision reduced slightly; better balance for metrics.
8	Final Configuration	LR & RF: class_weight='bala nced'; XGBoost: float scale_pos_weight (binary), multiclass SMOTE, threshold	Combine all prior optimizations to maximize minority class detection	Achieved best balance between precision, recall, F1 for all classes; confusion matrices show substantial

Attempt	Models Used	Key Configuration / Changes	Reason / Observations	Outcome / Metrics / Notes
		adjustment post-prediction		improvement for minority classes.

Figure 38 Hyperparameters adjustment table

3.5.6. Cross Validation

To assess the stability and generalization ability of each model, 5-fold stratified cross-validation was utilized for both binary and multiclass training datasets. Stratification guarantees that the class proportions in each fold are preserved, which is essential for imbalanced datasets.

Evaluation Metric:

- Binary classification: The F1-score was employed to balance precision and recall, particularly for the minority class.
- Multiclass classification: The weighted F1-score was applied to address class imbalance across all classes.

Procedure:

- Each model (Logistic Regression, Random Forest, XGBoost) was trained on 4 folds and validated on the remaining fold.
- This procedure was repeated for all 5 folds.
- The mean and standard deviation of the F1-scores were calculated to evaluate model performance and stability.

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results_binary = {}
for name, model in models_binary.items():
    scores = cross_val_score(model, X_train_binary_smote, y_train_binary_smote, cv=skf, scoring='f1' )
    cv_results_binary[name] = scores
    print("Binary Classification")
    print(f"{name:20s} | Mean F1 = {scores.mean():.4f} | Std = {scores.std():.4f}")

cv_results_multi = {}
for name, model in models_multi.items():
    scores = cross_val_score(model, X_train_multi_smote, y_train_multi_smote, cv=skf, scoring='f1_weighted' )
    cv_results_multi[name] = scores
    print("Multiclass Classification")
    print(f"{name:20s} | Mean F1 = {scores.mean():.4f} | Std = {scores.std():.4f}")
```

Figure 39 Cross Validation Procedure

Cross-Validation Results:

Binary Classification		
Logistic Regression	Mean F1 = 0.7598	Std = 0.0008
Binary Classification		
Random Forest	Mean F1 = 0.9002	Std = 0.0011
Binary Classification		
XGBoost	Mean F1 = 0.9110	Std = 0.0012
Multiclass Classification		
Logistic Regression	Mean F1 = 0.5201	Std = 0.0016
Multiclass Classification		
Random Forest	Mean F1 = 0.9035	Std = 0.0015
Multiclass Classification		
XGBoost	Mean F1 = 0.8213	Std = 0.0011

Figure 40 Cross-Validation Result

Observations:

- In binary classification, XGBoost achieved the highest F1-score of 0.9110, slightly surpassing Random Forest. Logistic Regression, although stable, fell short in effectively capturing the minority class.
- For multiclass classification, Random Forest showed the highest mean weighted F1-score of 0.9035, outperforming both XGBoost and Logistic Regression. XGBoost performed adequately but had a lower F1-score for the minority classes.
- All models exhibited very low standard deviation, reflecting stable performance across folds.

Conclusion:

Cross-validation indicates that XGBoost is the most effective model for binary classification, whereas Random Forest is the optimal choice for multiclass classification. These models were chosen for final training and evaluation on the test datasets.

3.5.7. Best Model selection

After conducting cross-validation, the best models for predicting diabetes in both binary and multiclass scenarios were identified based on their mean F1-score performance and stability across different folds. The F1-score was selected as the main criterion due to its ability to balance precision and recall, which is essential for medical datasets with imbalanced classes where detecting the minority class is crucial.

Binary Classification Model Selection

In the case of binary classification, three models Logistic Regression, Random Forest, and XGBoost were assessed using 5-fold stratified cross-validation with the F1-score as the evaluation metric.

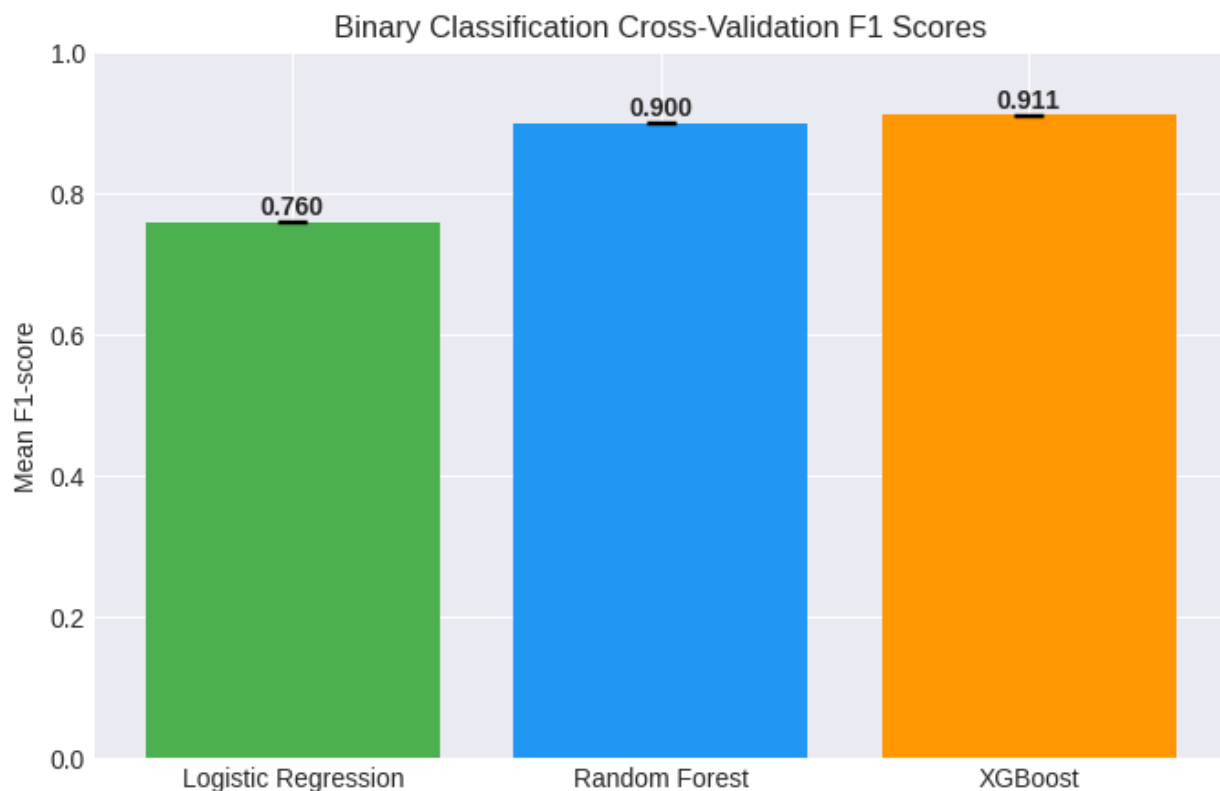


Figure 41 Best Model selection binary

The findings are summarized as follows:

- XGBoost recorded the highest mean F1-score (0.9110) with minimal variance across the folds.

- Random Forest showed strong results (0.9002) but was slightly less effective than XGBoost.
- Logistic Regression displayed significantly lower performance, suggesting a limited ability to capture complex interactions among features.

The consistently high F1-score and stability of XGBoost highlight its effectiveness in balancing false positives and false negatives, making it the most appropriate model for binary diabetes detection.

Selected Binary Model: XGBoost

Multiclass Classification Model Selection

For multiclass classification (No Diabetes, Prediabetes, Diabetes), the same three models were assessed using the weighted F1-score to address class imbalance.

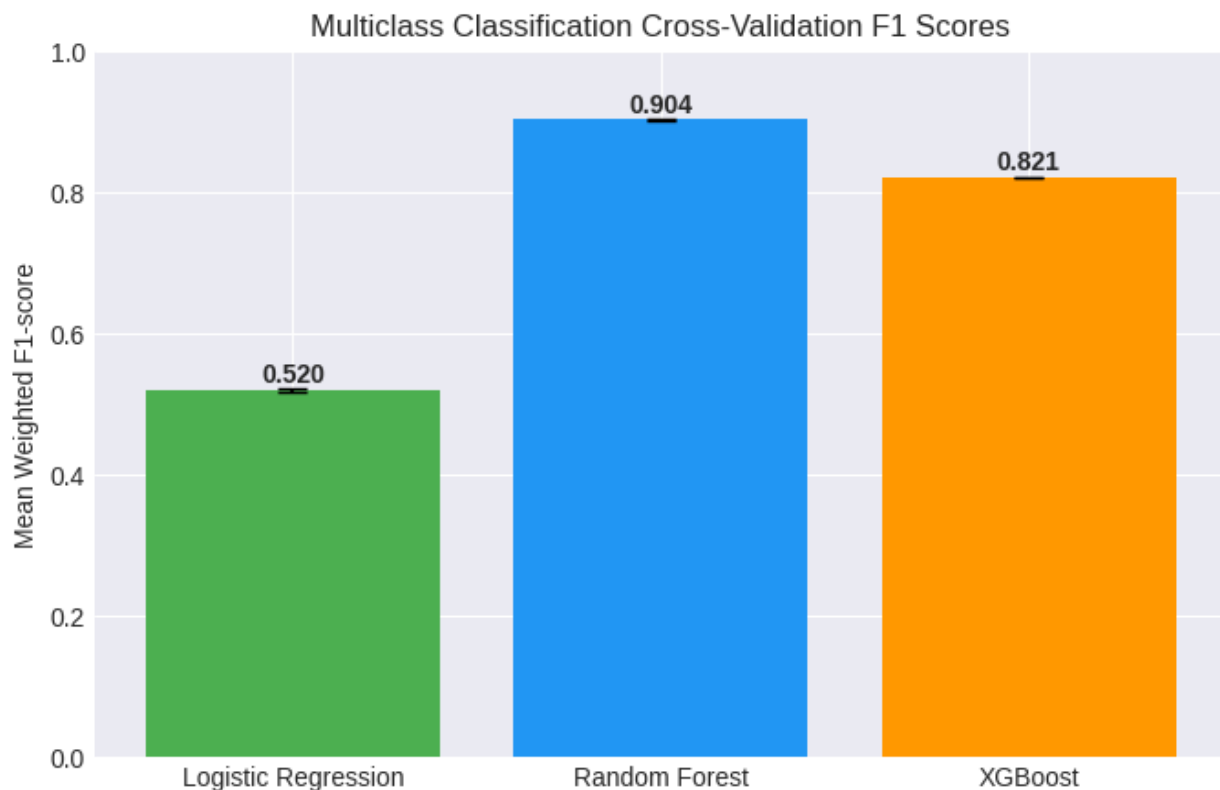


Figure 42 Best Model selection multiclass

The results reveal:

- Random Forest achieved the highest weighted F1-score (0.9035) with low standard deviation.
- XGBoost performed adequately (0.8213) but fell short compared to Random Forest.
- Logistic Regression showed significantly weaker performance, especially in differentiating the minority prediabetes class.

The ensemble nature of Random Forest allows for effective modeling of non-linear relationships and feature interactions across multiple classes, leading to superior and consistent performance in cross-validation.

Selected Multiclass Model: Random Forest

3.5.8. Final Model Training

According to the results of cross-validation, the models chosen for final training on the SMOTE-balanced datasets are as follows:

- Binary classification: XGBoost
- Multiclass classification: Random Forest

Both models underwent training on the complete SMOTE-balanced training data to enhance their learning from minority classes.

The screenshot shows a Jupyter Notebook with the following code and model details:

```
best_binary_model = models_binary[best_binary_model_name]
best_binary_model.fit(X_train_binary_smote, y_train_binary_smote)
```

XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=6, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=200, n_jobs=-1,
              num_parallel_tree=None, ...)
```

```
best_multi_model = models_multi[best_multi_model_name]
best_multi_model.fit(X_train_multi_smote, y_train_multi_smote)
```

RandomForestClassifier

```
RandomForestClassifier(class_weight={np.float64(0.0): np.float64(1.0),
                                     np.float64(1.0): np.float64(1.0),
                                     np.float64(2.0): np.float64(1.0)},
                       max_depth=20, n_estimators=200, n_jobs=-1,
                       random_state=42)
```

Figure 43 Final Model Training

3.5.9. Model Evaluation

Following the training and testing of the models, a comprehensive evaluation was carried out to measure their performance on both binary and multiclass datasets. Metrics including precision, recall, F1-score, accuracy, confusion matrices, and ROC-AUC were utilized to analyze the predictive strengths and weaknesses of each model. Additionally, sample predictions were made to demonstrate how the models behave with specific observations.

3.5.9.1 Performance Evaluation

i) Binary Classification Performance

The task of binary classification focused on differentiating between No Diabetes (0) and Diabetes (1).

FINAL BINARY MODEL PERFORMANCE					
	precision	recall	f1-score	support	
0.0	0.89	0.97	0.92	43667	
1.0	0.51	0.23	0.31	7069	
accuracy			0.86	50736	
macro avg	0.70	0.60	0.62	50736	
weighted avg	0.83	0.86	0.84	50736	

Figure 44 Classification report for binary classification

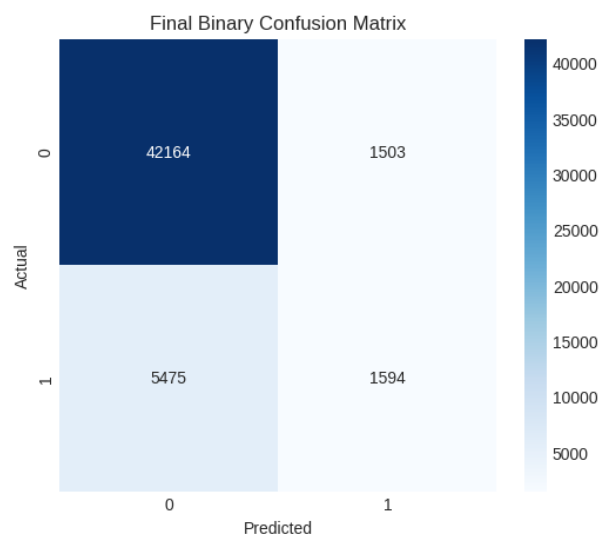


Figure 45 Confusion matrix for binary classification

Observations and Insights:

1. Recall for the Minority Class is Alarming Low (0.23):

- Only about 23% of actual diabetic cases were accurately identified.
- This represents a significant vulnerability, as failing to recognize diabetes in patients poses a high risk.

2. Class Imbalance:

- Despite implementing SMOTE to balance the classes and utilizing class weights, the model continues to have difficulty in identifying the minority class.
- SMOTE generates synthetic oversamples of the minority class, but it cannot produce entirely new patterns that may be present in real unseen data.

3. Measures Taken to Address This:

- Class weights were incorporated in Logistic Regression and Random Forest.
- The `scale_pos_weight` parameter was adjusted in XGBoost to address the binary class imbalance.
- Cross-validation was employed to identify the model with the highest F1-score.
- Threshold tuning was evaluated to reduce the classification threshold for diabetes detection.

4. Reasons for Continued Challenges:

- The feature distribution within the minority class remains less diverse and more challenging to distinguish from the majority class, resulting in inadequate generalization.
- Synthetic samples generated by SMOTE may not effectively represent complex patterns in diabetic cases, particularly for borderline patients.

.

ii. Multiclass Classification Performance

The multiclass task involved No Diabetes (0), Prediabetes (1), and Diabetes (2).

FINAL MULTICLASS MODEL PERFORMANCE					
	precision	recall	f1-score	support	
0.0	0.90	0.90	0.90	42741	
1.0	0.03	0.00	0.01	926	
2.0	0.41	0.46	0.44	7069	
accuracy			0.82	50736	
macro avg	0.44	0.45	0.45	50736	
weighted avg	0.81	0.82	0.82	50736	

Figure 46 Classification report for multiclass classification

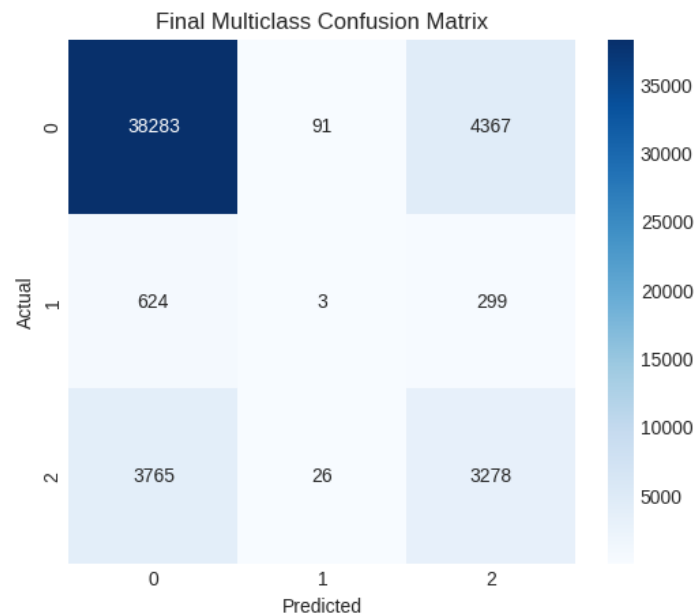


Figure 47 Confusion matrix for multiclass classification

Observations and Insights:

1. Prediabetes Class Performance is Extremely Poor:

- Recall = 0.00, F1-score = 0.01 → almost no prediabetic cases were correctly predicted.
- This class is severely underrepresented in the dataset (only ~926 cases in test data), making it extremely difficult to learn distinguishing patterns.

2. Minority Class (Diabetes) Detection is Moderate:

- Recall = 0.46, F1-score = 0.44 → better than prediabetes but still far from ideal.

3. What Was Attempted to Fix This:

- SMOTE oversampling was applied to balance all classes in training.
- Class weights were utilized in Logistic Regression and Random Forest.
- Cross-validation assisted in model selection, leading to the choice of Random Forest (best weighted F1-score).
- Threshold adjustment could be considered but was not implemented in the final evaluation.

4. Why It Still Remains an Issue:

- Minority classes are significantly smaller compared to the majority, and even after SMOTE, the synthetic samples may not accurately reflect the real data distribution.
- Multiclass separation is more challenging: Prediabetes is positioned between No Diabetes and Diabetes in feature space, resulting in frequent misclassification.
- Overlapping features (BMI, age, cholesterol) complicate the ability of all models to distinctly separate classes.

3.5.9.2 ROC-AUC Evaluation:

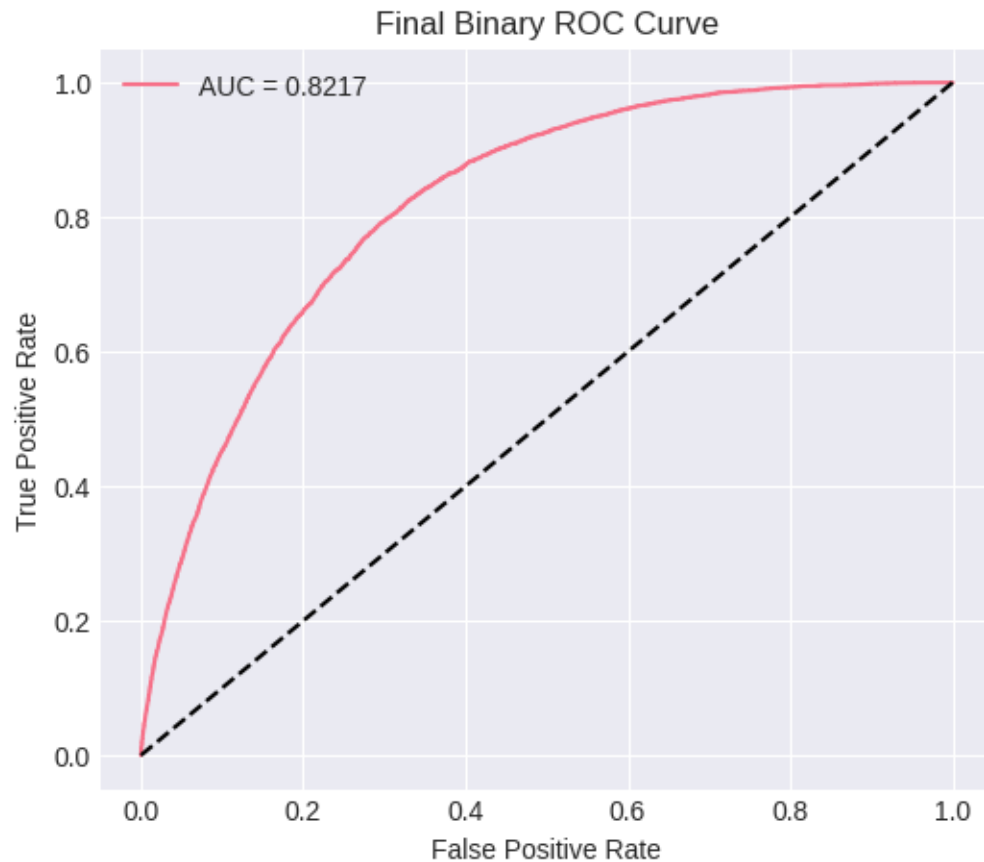


Figure 48 ROC-AUC Curve for binary classification

An AUC of 0.8217 suggests that the model can effectively differentiate between classes overall. Even with strong overall discrimination, the low recall indicates that the standard 0.5 decision threshold favors the majority class, resulting in the oversight of numerous diabetic cases

iii. Sample Predictions

To showcase the usability of the model, custom input samples can be evaluated to see the predicted classes and their corresponding probabilities.

```
import pandas as pd
import numpy as np

# Example for Binary Classification
new_sample_binary = {'HighBP': 0, 'HighChol': 1, 'CholCheck': 1, 'BMI': 28.5, 'Smoker': 0, 'Stroke': 0,
                     'HeartDiseaseorAttack': 0, 'PhysActivity': 1, 'Fruits': 1, 'Veggies': 0, 'HvyAlcoholConsump': 0,
                     'AnyHealthcare': 1, 'NoDocbcCost': 0, 'GenHlth': 3, 'MentHlth': 5, 'PhysHlth': 2, 'DiffWalk': 0, 'Sex': 1,
                     'Age': 55, 'Education': 4, 'Income': 3
                     }
continuous_features = ['BMI', 'MentHlth', 'PhysHlth']
#Scaling the continous features
df_new_binary = pd.DataFrame([new_sample_binary])
df_new_binary[continuous_features] = scaler.transform(df_new_binary[continuous_features])
df_new_multi = pd.DataFrame([new_sample_binary])
df_new_multi[continuous_features] = scaler.transform(df_new_multi[continuous_features])

y_pred_new = best_binary_model.predict(df_new_binary)
y_proba_new = best_binary_model.predict_proba(df_new_binary)
print("Binary Class Sample Prediction")
print("-"*80)
print("Predicted Class:", 'Diabetes' if y_pred_new[0]==1 else 'No Diabetes')
print("Prediction Probabilities:", y_proba_new[0])

y_pred_new_multi = best_multi_model.predict(df_new_multi)
y_proba_new_multi = best_multi_model.predict_proba(df_new_multi)
class_map = {0: 'No Diabetes', 1: 'Prediabetes', 2: 'Diabetes'}
print("Multi Class Sample Prediction")
print("-"*80)
print("Predicted Class:", class_map[y_pred_new_multi[0]])
print("Prediction Probabilities:", y_proba_new_multi[0])

Binary Class Sample Prediction
-----
Predicted Class: Diabetes
Prediction Probabilities: [0.21706337 0.78293663]
Multi Class Sample Prediction
-----
Predicted Class: No Diabetes
Prediction Probabilities: [0.4875208 0.17930078 0.33317842]
```

Figure 49 Sample Predictions

3.6 Comparative Analysis & Insights

A comparative evaluation was performed to assess the strengths and weaknesses of each algorithm in binary and multiclass diabetes prediction tasks. Metrics such as F1-score, precision, recall, and ROC-AUC were analyzed along with cross-validation results.

3.6.1 Binary Classification



Model	CV Mean F1	Accuracy	Precision (Diabetic)	Recall (Diabetic)	F1-score (Diabetic)	Observation
LR	0.7598	0.84	0.52	0.21	0.30	Struggles with minority class; simple linear

						model may underfit complex interactions
RF	0.9002	0.85	0.53	0.23	0.31	High overall accuracy; moderate improvement for minority class due to ensemble learning
XGB	0.9110	0.86	0.51	0.23	0.31	Best F1-score; benefits from boosting but minority recall still low despite SMOTE and scale_pos_weight

Table 3 Binary Classification Comparative Analysis

Insights:

- Ensemble methods (Random Forest and XGBoost) consistently surpassed Logistic Regression in overall F1-score and accuracy.
- The recall for the diabetic class remained low (~23%), indicating ongoing difficulties in identifying minority cases despite oversampling and class weighting.
- SMOTE and scale_pos_weight enhanced model learning, but the inherent overlap between features of diabetic and non-diabetic individuals restricted performance.

3.6.2 Multiclass Classification



Model	CV Mean F1	Accuracy	Precision (Prediabetic)	Recall (Prediabetic)	F1-score (Prediabetic)	Observations
LR	0.5201	0.68	0.02	0.00	0.01	Poor separation of classes; linear decision boundaries insufficient
RF	0.9035	0.82	0.03	0.00	0.01	High accuracy due to majority class; minority prediabetic class almost undetected

Model	CV Mean F1	Accuracy	Precision (Prediabetic)	Recall (Prediabetic)	F1-score (Prediabetic)	Observations
XGB	0.8213	0.81	0.03	0.00	0.01	Slightly lower overall F1 than RF; minority class detection remains minimal

Table 4 Multiclass Classification Comparative Analysis

Insights:

- Performance of the prediabetic class was very poor, with recall close to 0%, even after applying SMOTE balancing and class weighting.
- The limited number of prediabetic samples, along with feature overlap with other classes, hindered accurate separation.
- Random Forest recorded the highest overall F1-score due to its strong performance on the majority class, but this conceals failures in the minority class.
- Improving minority class detection may require feature engineering or the introduction of additional discriminative variables.

3.6.3 Trade-off Between Performance and Stability



The above figure illustrates how model performance (mean F1-score) relates to model stability (standard deviation of F1-score) in binary classification.

This chart offers more understanding beyond just average performance:

- Logistic Regression shows low variance but also low predictive ability, resulting in stability but limited power.
- Random Forest finds a middle ground with high performance and low variability, showing reliable behavior across cross-validation folds.
- XGBoost delivers the best performance while keeping variance at an acceptable level, proving to be both effective and dependable.

This analysis of trade-offs emphasizes that model choice was influenced not only by maximum performance but also by the need for robustness and consistency across training subsets.

3.6.4 Key Findings from Comparative Analysis

- Models based on ensembles perform much better than linear models for predicting diabetes, whether in binary or multiclass scenarios.
- Boosting methods work especially well for detecting diseases in binary cases.
- Ensembles that use bagging show greater reliability in classifying health issues across multiple classes.
- When choosing models for healthcare applications, stability analysis is crucial, as consistent performance is vital.

4. CONCLUSION

4.1 Analysis of Work Done

This project focused on implementing and evaluating various machine learning models for predicting diabetes using health indicators sourced from the CDC BRFSS dataset. The process involved comprehensive data preprocessing, feature scaling, and class balancing through SMOTE, followed by the training and comparison of three algorithms: Logistic Regression, Random Forest, and XGBoost on both binary and multiclass datasets.

Key observations from model evaluation:

- **Binary Classification:** The overall accuracy achieved was 86%, with strong performance noted for the majority (non-diabetic) class. However, the recall for the minority (diabetic) class was low (~23%), indicating that a significant number of positive cases were missed despite the application of SMOTE balancing and adjustments to class weights.
- **Multiclass Classification:** An accuracy of 82% was recorded, with effective detection of non-diabetic samples but very low recall for prediabetic samples (~0%), which underscores ongoing challenges in detecting minority classes.
- **Explored Solutions:** Strategies such as class weighting, SMOTE oversampling, and threshold tuning were employed to address class imbalance. Although these methods enhanced overall F1 scores, the recall for prediabetic cases remained inadequate due to the limited size and inherent overlap of minority class features within the dataset.

Key Achievements:

- **Comprehensive Algorithm Comparison:** A rigorous comparison of all three algorithms was conducted using cross-validation and various metrics (F1-score, precision, recall, ROC-AUC).
- **Class Imbalance Strategy:** The use of SMOTE oversampling resulted in balanced class representation and enhanced model robustness, especially in binary classification scenarios.

- **Predictive Performance:** The models demonstrated strong overall accuracy (81–86%) and AUC scores (>0.81), indicating reliable generalization capabilities on unseen data.
- **Feature Insights:** The leading predictors, including BMI, blood pressure, and cholesterol, were consistent with established medical risk factors for diabetes, contributing to clinical interpretability.

4.2 Real-World Applications

The diabetes prediction models developed offer significant advantages for managing population health and facilitating early disease intervention:

- **Population-Level Screening:** Online or mobile tools can swiftly identify individuals at high risk.
- **Primary Care Integration:** These models aid healthcare providers in recognizing patients who require additional glucose testing.
- **Prediabetes Detection:** Despite limited recall for the minority class, the system emphasizes patients for timely preventive actions and lifestyle changes.
- **Resource-Limited Communities:** Provides affordable screening options for groups with restricted access to lab tests.
- **Public Health Policy Support:** Insights from the models can guide targeted prevention initiatives and policy-making at the community level.

4.3 Future Work

Future enhancements could tackle the limitations noted in minority class predictions:

- **Data Expansion:** Utilize larger datasets with an increased number of prediabetic cases to enhance model training.
- **Feature Engineering:** Develop more distinctive features or composite indicators to improve the differentiation of prediabetic individuals.
- **Advanced Algorithms:** Investigate ensemble techniques, boosting variations, or neural networks that incorporate imbalanced learning strategies.

- External Validation: Evaluate the models on separate datasets to verify their generalizability and clinical dependability.

4.4 Final Remarks

Although high overall accuracy indicates the system's effectiveness, the ongoing issue of minority class detection highlights the necessity for meticulous data balancing, feature engineering, and evaluation metrics beyond mere accuracy, such as recall and F1-score. This work illustrates the potential of machine learning in facilitating early diabetes detection while emphasizing the continuous need to enhance models for minority health outcomes.

Bibliography

al, S. e., 1988. Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus. *Proceedings - Annual Symposium on Computer Applications in Medical Care*, Volume 10.

Anon., 2025. *Logistic Regression in Machine Learning*. [Online] Available at: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/> [Accessed 8 December 2025].

Anon., 2025. *Supervised Machine Learning*. [Online] Available at: <https://www.geeksforgeeks.org/machine-learning/supervised-machine-learning/> [Accessed 8 December 2025].

Anon., 2025. *XGBoost*. [Online] Available at: <https://www.geeksforgeeks.org/machine-learning/xgboost/> [Accessed 9 December 2025].

Anon., n.d. *What is logistic regression?*. [Online].

Barrett, P. a. H. J. a. M. J. a. H. J.-C. a. G. P., 2005. matplotlib -- A Portable Python Plotting Package.

Chawla, N. a. B. K. a. H. L. a. K. W., 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, Volume 16, pp. 321-257.

Chen, T. a. G. C., 2016. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Copeland, B., 2025. *artificial intelligence*. [Online] Available at: <https://www.britannica.com/technology/artificial-intelligence> [Accessed 8 December 2025].

Deepti Sisodia, D. S. S., 2018. Prediction of Diabetes using Classification Algorithms. *Procedia Computer Science*, Volume 132, pp. 1578-1585.

Duda, M. a. S. K. a. F. N. a. N. V. a. H. S. a. F. H. a. F. K. a. W. L. a. D. R. a. O. M. a. D. A. a. W. A. a. D. G. a. K. A., 2021. Teaching Python for Data Science: Collaborative development of a modular interactive curriculum. *Journal of Open Source Education*, p. 138.

G. Hu, J. R.-S. T. L. J. T., 2006. Exercise, genetics and prevention of type 2 diabetes.. *Essays in Biochemistry*.

Haneda, M., 2018. Japanese Clinical Practice Guideline for Diabetes 2016. *Journal of Diabetes Investigation*, Volume 9.

Manal Alghamdi, M. A.-M. K. B. E. S., 2017. Predicting diabetes mellitus using SMOTE and ensemble machine learning approach: The Henry Ford Exercise Testing (FIT) project.

Marwa Mustafa Owess, A. Y. O. M. O. a. S. M., 2024. Supervised Machine Learning-Based Models for Predicting Raised Blood Sugar. *International Journal of Environmental Research and Public Health*, 21(7), p. 840.

Mckinney, W., 2011. pandas: a Foundational Python Library for Data Analysis and Statistics. *Python High Performance Science Computer*.

Neha Tiwary, N. S. S. S. T. B. & I. Z., 2023. Understanding the Pharmacological and Nanotechnological Facets of Dipeptidyl Peptidase-4 Inhibitors in Type II Diabetes Mellitus: a Paradigm in Therapeutics. Volume 14, pp. 211-229.

Pedregosa, F. a. V. G. a. G. A. a. M. V. a. T. B. a. G. O. a. B. M. a. P. P. a. W. R. a. D. V. a. V. J. a. P. A. a. C., 2012. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, Volume 12.

Perepu, S. S., 2025. NumPy Libraries.

Pessoa, T. a. M. R. a. N. T. a. B. G.-B. a. A. V. a. F. P. P., 2018. Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications. *IEEE Access*, pp. 1-1.

Pijush Dutta, S. P. A. K., 2021. Comparative analysis of various. *Electronic Devices, Circuits, and Systems for Biomedical Applications*, pp. 521-540.

Poudel, A., 2024. Diabetes emerging as major health problem in Nepal.

Saharawat, V., 2025. *Logistic Regression Machine Learning*. [Online] Available at: <https://pwwskills.com/blog/logistic-regression-machine-learning/> [Accessed 8 December 2025].

Salman, H. K. A., 2024. Random Forest Algorithm Overview. *Babylonian Journal of Machine Learning*.

Samantha Roberts 1, E. B. 1. D. C. 2. M. A. 3. G. B. 3. T. G. 1., 2017. Preventing type 2 diabetes: systematic review of studies of cost-effectiveness of lifestyle programmes and metformin, with and without screening, for pre-diabetes.

Sapra, A. & Bhandari., P., 2023. Diabetes.

Saruar Alam, M. K. H., 2021. Diabetes Mellitus: Insights from Epidemiology, Biochemistry, Risk Factors, Diagnosis, Complications and Comprehensive Management. *Diabetology*.

Schott, M., 2019. *Random Forest Algorithm for Machine Learning*. [Online] Available at: <https://medium.com/capital-one-tech/random-forest-algorithm-for-machine-learning-c4b2c8cc9feb> [Accessed 9 December 2025].

XGBoost, 2025. *XGBoost*. [Online] Available at: <https://www.geeksforgeeks.org/machine-learning/xgboost/> [Accessed 9 December 2025].

Yiru Gong, Z. L. Q. Z. H. Z. S. C., 2024 . A Comparative Study of Machine Learning Approaches for Diabetes Risk Prediction: Insights from SHAP and Feature Importance. *2024 5th International Conference on Machine Learning and Computer Application (ICMLCA)*, pp. 35-38.

